UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY



Research and Development

By

Đặng Đình Hòa 23BI14169

Major: Data Science

Title:

Intelligent Weather-Aware Travel Assistant for Vietnam

External Supervisor: MSc. Vũ Hà My
Internal Supervisor: Dr. Nghiêm Thị Phương

Hanoi, June 2026

Contents

List of Tables **4**

List of Figures **5**

Acknowledgements **6**

List of Abbreviations **7**

Abstract **9**

1 Introduction and Problem Statement **10**
1.1 Background and Motivation 10
1.2 Target Users and Their Needs 10
1.3 Problem Statement and Project Objectives 11
1.4 Main Functions and User Journey 11
1.4.1 Checking Weather for a Destination 12
1.4.2 Comparing Routes Before Departure 12
1.4.3 Returning to Frequently Used Information 12
1.5 Scope and Limitations 13

2 Theoretical Foundations and Core Technologies **14**
2.1 Spatial Data for the Web Application 14
2.2 Weather Data and Freshness 14
2.3 Core Route and Risk Algorithms 15
2.3.1 Route Sampling with Estimated Arrival Time 15
2.3.2 Checkpoint Risk Assessment 15
2.3.3 Comparing Route Alternatives 15
2.4 Core Web Technologies 16
2.5 Supporting Assistant 16
2.6 Use Case Model and Requirements 16
2.7 Overview of the System Architecture 19

3 System Architecture and Detailed Design **21**
3.1 System Architecture Overview and C4 Container Model 21
3.1.1 Microservice Architecture and Component Decomposition 21
3.1.2 Network Segmentation and Zero-Trust Isolation Policies (Docker Segmentation) 23
3.2 Secure Session Boundary and Identity Architecture (ADR-004) 23
3.2.1 Token Exfiltration Vulnerability Analysis and BFF Boundary Architecture 23

3.2.2 AES-256-GCM Authenticated Encryption Envelope and Opaque Session Handles . . . 23
3.2.3 Unified Session Lifecycle: Issuance, Sliding Rotation, and Synchronized Revocation . . . 24
3.3 Background Integration Worker and Resilience Patterns (ADR-001) . . . 30
3.3.1 Fault Domain Isolation Between Background Tasks and APIs . . . 30
3.3.2 Distributed Database Leasing and Concurrency Control . . . 30
3.3.3 Circuit Breaker Pattern and Adaptive Backoff . . . 30
3.3.4 Meteorological Data Freshness Lifecycle and Safe States . . . 31
3.4 PostGIS Spatial Database Schema and Entity-Relationship Architecture . . . 33
3.4.1 Tiered Database Domain Architecture . . . 33
3.4.2 Spatial Query Optimization with GiST Indexes . . . 33
3.4.3 Standardized Entity-Relationship Diagram and Governance Constraints . . . 34
3.5 Multi-Objective Route Risk Evaluation and Balancing Engine . . . 35
3.5.1 Spatiotemporal Modeling and Route Corridor Discretization . . . 35
3.5.2 Vehicle-Differentiated Hazard Threshold Matrix . . . 35
3.5.3 Balanced Multi-Objective Route Optimization Function . . . 36
3.5.4 Route Hazard Evaluation Pipeline and Sequence Diagram . . . 36
3.6 Grounded Conversational Weather AI Engine . . . 38
3.6.1 Tool Orchestration Architecture . . . 38
3.6.2 Grounded Evidence Pipeline and Citation Synthesis . . . 38

4 System Implementation **39**
4.1 Development Environment and Deployment Stack . . . 39
4.1.1 Monorepo Architecture and Static Type Contracts . . . 39
4.1.2 Three-Tier Network Partitioning and Containerization . . . 39
4.2 Backend Service Layer and Session Security Boundary Implementation (ADR-004) 40
4.2.1 Fastify 5 Request Lifecycle and Hook Pipeline . . . 40
4.2.2 Atmospheric Cryptographic Envelope and Timing-Safe Protections . . . 40
4.2.3 Durable Session Store and Session Family Annihilation (ADR-004) . . . 41
4.3 PostGIS Spatial Database and GiST Query Optimization . . . 42
4.3.1 Spatial Schema Architecture and GiST Index Declarations . . . 42
4.3.2 Two-Tier Spatial Corridor Query Specification . . . 42
4.4 Background Integration Worker and Resilience Engine Implementation (ADR-001) 43
4.4.1 Distributed PostgreSQL Database Leasing Mechanism . . . 43
4.4.2 Three-State Circuit Breaker and Exponential Backoff with Jitter . . . 44
4.4.3 WMO-No. 8 Normalization and SHA-256 Cryptographic Deduplication . . . 44
4.5 Route Hazard Evaluation and Multi-Alternative Planning Implementation . . . 45
4.5.1 OSRM Routing Orchestration and Spatiotemporal Discretization . . . 45
4.5.2 Vehicle-Differentiated Risk Matrix Evaluation and Multi-Objective Scoring 45
4.6 Grounded Conversational Weather AI Assistant and Fallback Adapter . . . 45
4.6.1 Structured Tool Orchestration with Gemini 1.5 Flash . . . 45
4.6.2 Grounded Verification Pipeline and Citation Synthesis . . . 46
4.6.3 Deterministic Fallback Adapter Implementation . . . 46
4.7 Web Frontend and 3D Geospatial Visualization Implementation . . . 47
4.7.1 Frontend Architecture with React 18 and Tiered State Management . . . 47
4.7.2 WebGL 3D Geospatial Visualization with MapLibre GL . . . 47
4.8 Software Testing Strategy, Quality Assurance, and CI/CD Pipeline . . . 48

4.8.1 Four-Tier Testing Pyramid 48
4.8.2 Automated Continuous Integration and Deployment Pipeline 48

5 Experimental Evaluation and Results **49**
5.1 Evaluation Scope and Test Setup 49
5.2 Implemented Functions and Route Results 49
5.2.1 Vehicle-Differentiated Risk Results 50
5.2.2 Route Planner Interface 50
5.3 System Performance and Reliability 50
5.3.1 PostGIS and GiST Spatial Queries 50
5.3.2 Fastify API, Redis, and Worker Reliability 51
5.4 Grounded Assistant Results 51
5.5 Web Application and Usability Results 52
5.5.1 System Usability Scale 54
5.6 Achievement Against Project Objectives 54

6 Conclusion and Future Work **55**
6.1 Summary of Project Achievements 55
6.2 Goal Fulfillment Matrix 55
6.3 Current Technical Limitations 57
6.4 Future Research and Development Roadmap 57
6.4.1 Short-Term Roadmap 57
6.4.2 Medium-Term Roadmap 57
6.4.3 Long-Term Roadmap 57
6.5 Concluding Remarks 57

References **58**

List of Tables

1.1 Main user functions of VietForecast. 12

2.1 Use case overview and primary requirements. 17

3.1 Vehicle-differentiated weather hazard activation matrix 36

4.1 PostGIS corridor spatial query technical specifications 43
4.2 Allowlisted Gemini Function Calling tool specifications 46

5.1 Quantitative performance comparison of multi-alternative routing profiles 49
5.2 PostGIS spatial corridor query latency benchmarks 51
5.3 Quantitative evaluation of grounded AI assistant across complex query tiers . . . 51
5.4 System Usability Scale survey results across 10 standardized criteria 54

6.1 System goal fulfillment and benchmarking matrix 56

List of Figures

2.1 Overview of the VietForecast use cases. 17
2.2 High-level system context architecture. 20

3.1 C4 Container Diagram (Level 2) of VietForecast 22
3.2 Sequence diagram of session lifecycle (ADR-004) - Phase 1: Session Issuance . . 25
3.2 Sequence diagram of session lifecycle (ADR-004) - Phase 2: Sliding Rotation . . 27
3.2 Sequence diagram of session lifecycle (ADR-004) - Phase 3: Synchronized Revocation 29
3.3 Sequence diagram of background meteorological ingestion (ADR-001) 32
3.4 Entity-Relationship Diagram (ERD) of VietForecast 34
3.5 Sequence diagram of multi-objective route hazard evaluation 37

5.1 Production Route Risk Planner interface: Weather-Safer alternative 50
5.2 Conversational AI assistant interface handling complex routing inquiries 52
5.3 Interactive nationwide weather map across 63 provinces 53
5.4 Dedicated Nationwide Weather Directory (63 Provinces) 53

Acknowledgements

I would like to express my sincere gratitude to all those who gave me the opportunity to complete this thesis in particular, as well as the comprehensive academic support that I received throughout my studies at the University of Science and Technology of Hanoi (USTH).

Foremost, I would like to send my profound thanks to my supervisors, MSc. Vũ Hà My (External Supervisor) and Dr. Nghiêm Thị Phương (Internal Supervisor), for their patient and dedicated guidance, valuable insights, and continuous encouragement, which enabled me to successfully complete this thesis. Their invaluable support not only helped me improve my technical and research capabilities but also taught me essential professional skills during my project and internship under their supervision.

I would also like to convey my sincere gratitude to all the lecturers in the Department of Information and Communication Technology (ICT) for accompanying and mentoring me throughout my time at the university. Their dedication has helped me build a solid knowledge foundation in computer science and data science, giving me the confidence to pursue my chosen career path.

Finally, I wish to express my deepest appreciation to my family and friends for their unwavering patience, encouragement, and support throughout this journey.

List of Abbreviations

ADR Architecture Decision Record

AI Artificial Intelligence

API Application Programming Interface

BBOX Minimum Bounding Box

BFF Backend-for-Frontend

CI/CD Continuous Integration and Continuous Deployment

CMS Content Management System

CSRF Cross-Site Request Forgery

EPSG European Petroleum Survey Group (Geodetic Parameter Registry)

ETA Estimated Time of Arrival

GeoJSON Geographic JavaScript Object Notation

GiST Generalized Search Tree

GPS Global Positioning System

GPU Graphics Processing Unit

HTTP Hypertext Transfer Protocol

HTTPS Hypertext Transfer Protocol Secure

I/O Input/Output

JSON JavaScript Object Notation

LLM Large Language Model

OSRM Open Source Routing Machine

PostGIS PostgreSQL Spatial Database Extender

RBAC Role-Based Access Control

RFC Request for Comments

SQL Structured Query Language

SUS System Usability Scale

TTL Time-To-Live

UI User Interface

VU Virtual User

WebGL Web Graphics Library

WGS84 World Geodetic System 1984

WMO World Meteorological Organization

XSS Cross-Site Scripting

Abstract

Vietnam possesses a long and narrow geography characterized by complex tropical monsoon climates and frequent severe weather phenomena such as typhoons, torrential rains, dense fog, and thunderstorms. These meteorological factors pose critical travel safety risks, particularly for motorcyclists, who constitute the predominant proportion of commuters in the country.

This graduation project presents the research, design, and implementation of the "Smart Vietnam Weather Information and Safe Travel Route Planning System with AI Grounding" (VietForecast). The system addresses core technical challenges through the following key contributions:

1. Meteorological Ingestion and Spatial Normalization: Automated real-time harvesting across 63 Vietnamese provinces, persisted within a PostGIS spatial database with continuous validation of data authenticity, integrity, and temporal freshness.
2. Multi-Dimensional Weather Risk Evaluation Engine: A config-driven matrix evaluating hazards across precipitation intensity, wind gusts, surface visibility, extreme temperatures, and road surface conditions, customized by vehicle profile (motorcycles and automobiles).
3. Multi-Alternative Weather-Aware Route Optimization: Concurrent calculation of Fastest, Shortest, and Weather-Safer routes for inter-provincial corridors spanning up to 2,000 km.
4. Grounded AI Travel Assistant: Integration with Google Gemini LLM via function calling over internal tool declarations, ensuring responses are strictly grounded with verifiable spatial citations, drastically suppressing hallucinations.
5. Containerized Architecture and Quota Governance: Segmented Docker networking, atomic Redis quota counters aligned with Asia/Ho Chi Minh midnight resets, and integrated digital payments.

Keywords: Weather-Aware Routing, Travel Risk Assessment, PostGIS, LLM Grounding, Fastify, MapLibre GL, Vietnam Meteorological Systems.

Chapter 1

Introduction and Problem Statement

1.1 Background and Motivation

Weather is an everyday concern for people travelling in Vietnam. Rain, strong winds, heat, and fog can affect visibility, comfort, and journey time. These conditions also influence road safety [1], particularly for motorcycle riders who are directly exposed to the weather. On longer journeys, conditions at the destination may differ considerably from those along the route.

Travellers can consult navigation applications for directions and weather applications for forecasts. However, planning a journey often requires switching between these services and checking several locations separately. A forecast for the destination alone does not explain what a traveller may encounter on the way. This makes it difficult to compare routes while considering both travel time and weather conditions.

VietForecast addresses this practical need through a web application that brings weather information, an interactive map, and route comparison into one place. Its purpose is to help users understand the available information and make better-informed travel decisions before departure.

1.2 Target Users and Their Needs

The application is intended primarily for individual travellers in Vietnam. Three user groups guide its design:

- **Motorcycle riders and commuters** need to check rain, wind, and visibility before travelling. Weather information is especially useful when deciding whether to depart, postpone a trip, or consider another route.
- **Car drivers and families** need an overview of conditions along inter-provincial journeys, together with route distance and estimated travel time. They can use this information to compare the convenience and weather exposure of available alternatives.
- **Tourists and occasional travellers** need an accessible way to explore unfamiliar destinations and check forecasts without interpreting technical meteorological data.

These groups share a need for clear information about *where*, *when*, and *how* to travel. The interface therefore prioritizes location search, readable weather summaries, and route comparison.

Administrators form a separate operational user group responsible for managing accounts and inspecting the data and services that support the application.

1.3 Problem Statement and Project Objectives

The problem addressed by this project is how to make weather information useful within the process of planning a road journey. A user should be able to select a location or enter an origin and destination, choose a departure time and vehicle type, and understand the conditions relevant to that journey through a single web interface.

The main objective is to develop a usable web application that connects weather exploration with travel planning. Its specific objectives are to:

1. Present current conditions and available forecasts in a consistent, easy-to-read format.
2. Help users explore weather across Vietnam through a searchable directory and an interactive map.
3. Compare route alternatives using distance, estimated travel time, and weather assessments for cars and motorcycles.
4. Make data freshness, missing information, and assessment limitations visible to users.
5. Support returning users through accounts, saved locations, saved routes, and interface preferences.

The project's main contribution is the integration of these activities into one application. A conversational assistant provides an additional way to ask weather and travel questions; the map, weather pages, and route planner remain the core of the user experience.

1.4 Main Functions and User Journey

Table 1.1 summarizes the main functions in terms of what users can do and the benefit each function provides.

Table 1.1: Main user functions of VietForecast.

Function	User action and benefit
Weather lookup	Search for a province or city, inspect current conditions, and view available forecasts to prepare for a visit or journey.
Interactive map	Select a location and a forecast time to explore weather geographically and open local details.
Route planning	Enter an origin, destination, departure time, and vehicle type; compare fastest, shortest, and weather-safer alternatives.
Weather assessment	Inspect route checkpoints, weather warnings, and data coverage to understand why an alternative is recommended or remains uncertain.
Accounts and preferences	Register or sign in, manage saved locations and routes, and review account settings and plan limits.
Conversational assistance	Ask weather and travel questions through a chat interface with references to the information used in the response.

1.4.1 Checking Weather for a Destination

A user opens the weather page or map and searches for a destination. The application displays available measurements, such as temperature, rainfall, wind, humidity, and visibility. Selecting a forecast time updates the information, while opening the location details provides a closer view of forecast trends. For example, a tourist considering a visit to Da Nang can check the expected conditions before choosing a day to travel.

1.4.2 Comparing Routes Before Departure

A user enters the starting point and destination, chooses a car or motorcycle, and selects a departure time. The route planner displays available alternatives with their distance, estimated duration, and weather information along the journey. The user can select an alternative on the map and inspect its checkpoints and warnings.

For example, a driver planning a trip from Hanoi to Lao Cai can compare the time and distance of the suggested routes alongside their weather assessments. If weather coverage is incomplete, the interface identifies that limitation so the recommendation can be interpreted with appropriate caution. The available alternatives may share road sections or use the same geometry; they are not guaranteed to be three distinct routes.

1.4.3 Returning to Frequently Used Information

Users can browse the public weather and route pages, while registered users can access personal settings and manage saved locations and routes. Vietnamese and English interfaces, together with light and dark themes, support different reading preferences. Account plans define usage limits for selected services. The assistant offers a supplementary entry point for questions, while the main pages provide the detailed information needed for comparison.

1.5 Scope and Limitations

The project focuses on a browser-based application for road travel within Vietnam, supporting cars and motorcycles. The current dataset uses a 63-province location catalogue. This describes the dataset used by the project rather than a claim that it reflects the latest administrative boundaries. Available forecasts extend up to 72 hours, depending on provider coverage and data freshness.

Weather assessments depend on the measurements available for a location and time. Missing or outdated evidence must be distinguished from low-risk conditions. A weather-safer recommendation represents a comparison of available alternatives under the application's rules; it does not guarantee a safe journey or account for every road hazard. Weather information obtained from external providers is not presented as a direct connection to a national observation network.

The application supports planning and information lookup. Live traffic management, emergency response, and a dedicated mobile turn-by-turn navigation service are outside the project scope. Drivers remain responsible for following road restrictions, official warnings, and conditions observed during travel. Commercial fleet management and the development of a new weather forecasting model are also outside the scope.

Chapter 2

Theoretical Foundations and Core Technologies

This chapter introduces the concepts needed to understand the VietForecast web application. It focuses on the data representations and algorithms that support weather lookup, map exploration, and route comparison. Detailed implementation and deployment decisions are discussed in later chapters.

2.1 Spatial Data for the Web Application

VietForecast represents locations and routes using the WGS84 geographic reference system (EPSG:4326) and the GeoJSON format defined by RFC 7946 [2]. A weather location is represented as a point, a road journey as a LineString, and an administrative area as a polygon. GeoJSON allows the API and the frontend map to exchange these objects using the same structure.

Weather and route data are stored in PostgreSQL with the PostGIS extension [3]. PostGIS provides spatial types and queries for finding nearby stations, checking whether a point lies within a route corridor, and calculating distances. A GiST spatial index, based on the spatial-tree approach used for indexed search, reduces the number of records that must be examined before an exact distance or intersection check is performed [4]. This gives the application a practical way to search weather evidence near a route without scanning the complete dataset.

2.2 Weather Data and Freshness

The application uses normalized weather observations and forecasts. Provider-specific condition codes are mapped to a small set of application states, such as clear, cloudy, rain, fog, and thunderstorm. Measurements are kept with their source time and freshness state so that the interface can distinguish current, aging, stale, and unavailable data. The data model follows the general observation principles described by WMO-No. 8 [5].

Freshness is part of the user-facing result. A missing or outdated measurement is reported as unavailable or unknown rather than being silently treated as safe weather. This rule is important for both the weather pages and the route planner because a recommendation is only as reliable as the

evidence covering the relevant location and time.

2.3 Core Route and Risk Algorithms

2.3.1 Route Sampling with Estimated Arrival Time

The routing provider returns a road geometry, distance, and estimated duration. Road-network routing and OpenStreetMap-based services provide the underlying route alternatives [6–8]. The application samples the geometry at bounded spatial and temporal intervals. For each checkpoint p_i, it estimates an arrival time from the departure time t_0 and the distance travelled before that checkpoint:

$$t_i = t_0 + \sum_{j=1}^{i} \frac{d(p_{j-1}, p_j)}{\bar{v}}, \tag{2.1}$$

where $\bar{v}$ is the average route speed. The checkpoint location and estimated time are then used to find the weather evidence that is relevant to the vehicle at that point in the journey. This is the main difference between checking weather at the destination and checking weather along a route.

2.3.2 Checkpoint Risk Assessment

Each checkpoint is assessed using the available weather dimensions: rainfall, wind gust, temperature, visibility, storm information, and official alerts. Each dimension is mapped to a simple severity level: Low, Caution, High, or Unknown. The checkpoint level is the highest applicable severity among the available dimensions. If required evidence is missing or stale, the result remains Unknown and the response includes a warning.

The route summary reports the highest checkpoint severity, the proportion of covered checkpoints, and a distance-aware summary of the available evidence. These values are shown together with the warnings and source information so that users can understand both the result and its limitations. The assessment is vehicle-aware: the same weather conditions can produce different warnings for a motorcycle and a car because their exposure and stability differ.

2.3.3 Comparing Route Alternatives

The route planner compares alternatives returned by the routing service, typically labelled Fastest, Shortest, and Weather-Safer. The comparison considers three practical factors:

- travel time;
- route distance; and
- weather risk and evidence coverage.

The Weather-Safer label is assigned from the application's balanced route score. It is a relative comparison between the available alternatives, not a guarantee that a journey is safe. The interface therefore presents the route geometry, score, warnings, and evidence coverage together instead of displaying a single unexplained number.

2.4 Core Web Technologies

The project uses a TypeScript monorepo with React and Vite for the frontend and Fastify for the API. PostgreSQL/PostGIS stores weather, location, and route-related data, while Redis supports short-lived caching and usage limits [9]. A background worker obtains and normalizes provider data so that the API can serve the web application without performing every external request during a page interaction.

The frontend is a single-page web application. React manages page and component state, while MapLibre GL [10] renders the interactive map, weather layers, route lines, and checkpoint markers. The main user-facing pages are the weather directory, location detail view, map, and route planner. Account settings, saved locations, saved routes, and plan limits support returning users.

Fastify [11] exposes typed API endpoints for weather, forecasts, routes, risk assessments, accounts, and chat. Directus provides the administration and content-management services used by the application [12]. Shared TypeScript contracts keep the request and response shapes consistent between the frontend and backend. Docker Compose packages the web application, API, worker, database, and cache for local or server deployment.

2.5 Supporting Assistant

The conversational assistant is an optional interface for asking weather and travel questions. It uses a small set of allowlisted operations, such as location search, current weather lookup, forecast lookup, risk assessment, and route evaluation. The optional Gemini integration is treated as a language interface over these backend tools rather than as an independent source of weather facts [13]. The backend executes these operations and returns the resulting data and citations to the interface.

The assistant complements the map, weather pages, and route planner; it does not replace them. The core application remains usable without a conversational model, and responses must indicate when evidence is missing or unavailable.

2.6 Use Case Model and Requirements

The application has three actor groups. Guest users can view weather and compare routes. Registered users have the same access plus saved routes, saved locations, and personal settings. Administrators maintain accounts, monitor data services, and publish official alerts. The following overview keeps the complete functional boundary while leaving implementation details to Chapters 3 and 4.

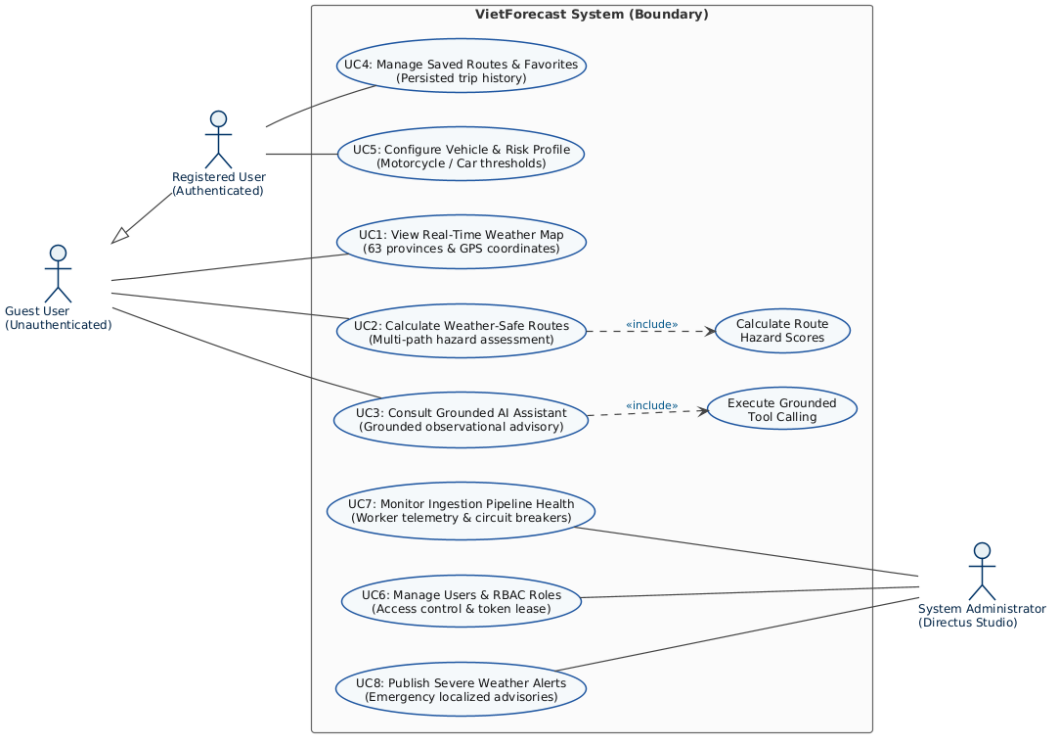


Figure 2.1: Overview of the VietForecast use cases.

Table 2.1: Use case overview and primary requirements.

ID	Use case	Actor	Required result
UC1	View weather map	Guest, registered	Search a location and display current or forecast weather on the map.
UC2	Calculate weather-aware routes	Guest, registered	Return route alternatives with time, distance, risk, warnings, and coverage.
UC3	Consult assistant	Guest, registered	Answer weather and travel questions from available application evidence.
UC4	Manage saved routes and locations	Registered	Save, view, and remove personal travel information.
UC5	Configure travel profile	Registered	Store vehicle type and applicable preferences for later planning.
UC6	Manage users and roles	Administrator	Maintain accounts and access permissions.
UC7	Monitor data services	Administrator	Inspect worker status, provider freshness, and service availability.
UC8	Publish weather alerts	Administrator	Create and publish an alert for an affected area.

UC1: View Weather Map

- Actors: Guest or registered user.
- Precondition: The web client and weather API are available.
- Main flow: The user searches for a location or selects a map point and, if needed, a forecast time. The system finds the location and displays temperature, rainfall, wind, humidity, visibility, and freshness information.
- Postcondition: The selected location and weather state are visible on the map and detail view.
- Exception: If the location or weather evidence cannot be resolved, the interface shows an unavailable or unknown state and explains the limitation.

UC2: Calculate Weather-Aware Routes

- Actors: Guest or registered user.
- Precondition: A valid origin, destination, departure time, and vehicle type are provided.
- Main flow: The system requests route alternatives, samples each route, estimates checkpoint arrival times, retrieves nearby weather evidence, and assigns checkpoint risk levels. It returns fastest, shortest, and weather-safer comparisons with route lines, warnings, and evidence coverage.
- Postcondition: The user can select an alternative and inspect its checkpoints and assessment.
- Exception: If routing or weather services fail, the system retries where applicable and reports that the route or assessment is incomplete.

UC3: Consult the Supporting Assistant

- Actors: Guest or registered user.
- Precondition: The user submits a weather or travel question through chat.
- Main flow: The assistant identifies the requested operation, such as location search, forecast lookup, risk assessment, or route evaluation. The backend executes the operation and returns the result and source information.
- Postcondition: The user receives a concise answer linked to available application evidence.
- Exception: If the location or requested information is ambiguous or unavailable, the assistant asks for clarification or states that no reliable result is available.

UC4: Manage Saved Routes and Locations

- Actor: Registered user.
- Precondition: The user has an authenticated session.
- Main flow: The user saves a route or location, views the saved list, or removes an item. The API validates ownership and updates the user's records.
- Postcondition: The saved information is available on the user's account and can be reused in later visits.
- Exception: If the session expires or validation fails, the change is rejected and the user is asked to sign in again or correct the input.

UC5: Configure the Travel Profile

- Actor: Registered user.
- Precondition: The user is signed in and opens the account preferences.
- Main flow: The user selects a motorcycle or car profile and updates supported preferences. The system validates and stores the values for future route assessments.
- Postcondition: Later route results use the selected vehicle context.
- Exception: Invalid or unsupported values are rejected with a correction message.

UC6: Manage Users and Roles

- Actor: System administrator.
- Precondition: The administrator is authenticated with the required privileges.
- Main flow: The administrator views accounts, changes permitted account or role states, and reviews the resulting records.
- Postcondition: Account access reflects the approved administrative change.
- Exception: Unauthorized changes are blocked and recorded according to the administration rules.

UC7: Monitor Data Services

- Actor: System administrator.
- Precondition: The monitoring view and service telemetry are available.
- Main flow: The administrator checks worker activity, provider response state, cache or data freshness, and recent failures.
- Postcondition: The current operational state and stale data conditions are visible for follow-up.
- Exception: If telemetry is unavailable, the dashboard reports the monitoring gap instead of presenting an outdated status as current.

UC8: Publish Weather Alerts

- Actor: System administrator.
- Precondition: An official or reviewed warning is available with a location and severity.
- Main flow: The administrator enters the alert text, hazard type, severity, and affected area, then validates and publishes it. The application shows the alert in the relevant user views.
- Postcondition: Users can see the published warning for the affected area.
- Exception: Missing text, severity, or spatial information prevents publication until corrected.

2.7 Overview of the System Architecture

At context level, VietForecast contains a browser client, an application API, persistent and background services, and external providers. The browser presents the weather directory, map, route planner, account pages, and optional assistant. The API validates requests and coordinates weather lookup, routing, risk assessment, saved data, and chat operations. PostgreSQL/PostGIS stores application and spatial data, Redis supports short-lived caching and limits, and a worker normalizes provider data. Weather and routing providers supply external observations, forecasts, and road ge-

ometry. The diagram defines these boundaries; detailed components and deployment are described in Chapters 3 and 4.

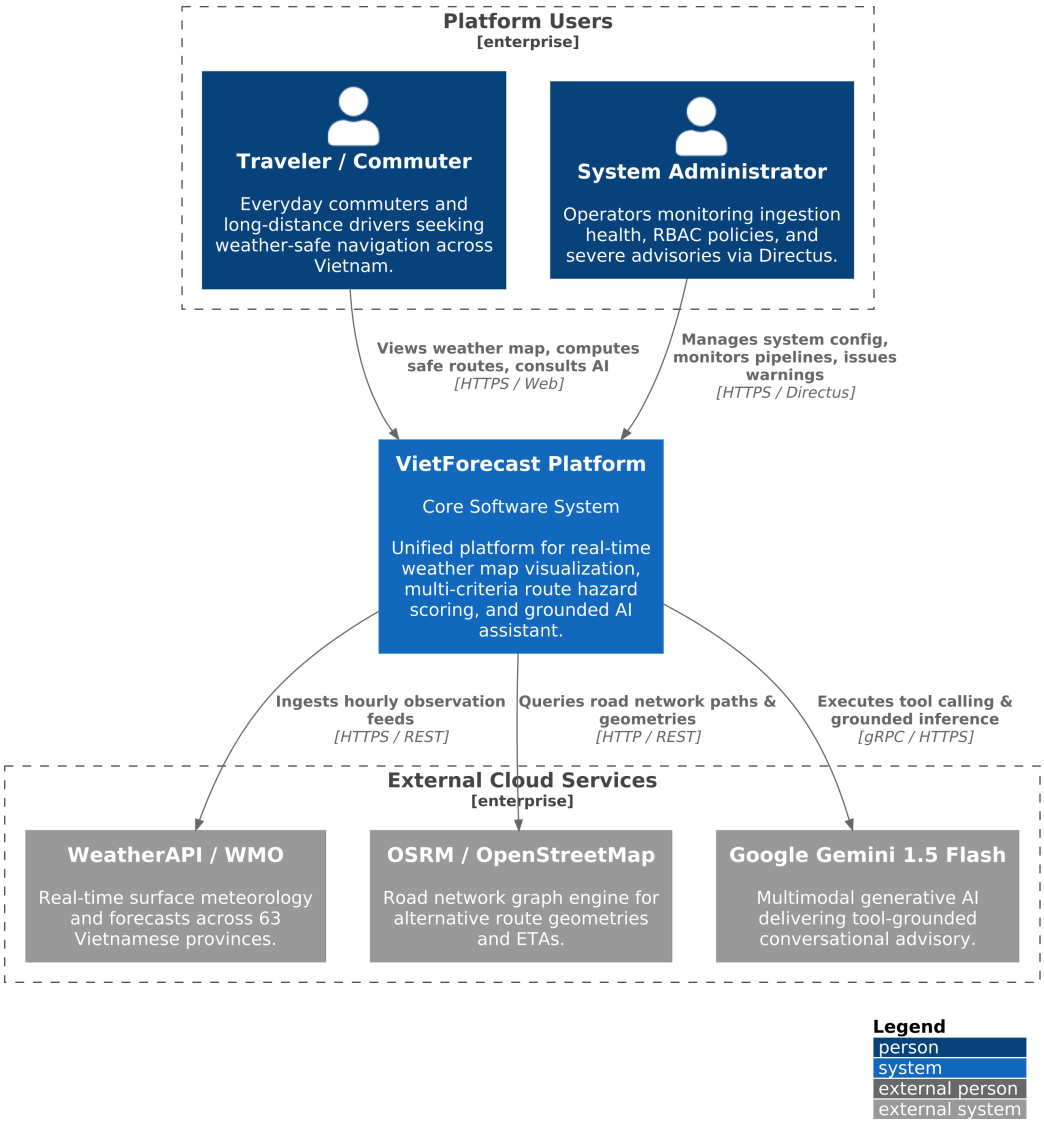


Figure 2.2: High-level system context architecture.

Chapter 3

System Architecture and Detailed Design

This chapter describes the VietForecast architecture, security boundaries, background services, spatial database, route-risk engine, and supporting assistant. The design separates web requests from scheduled data ingestion, protects backend secrets, and keeps weather evidence traceable.

3.1 System Architecture Overview and C4 Container Model

3.1.1 Microservice Architecture and Component Decomposition

VietForecast uses containerized services so interactive web requests and scheduled weather ingestion run independently. The frontend, API, worker, database, cache, and administration services communicate through defined internal interfaces.

Figure 3.1 depicts the C4 Model Level 2 (Container Diagram) of the VietForecast platform, delineating network boundaries, containerized execution units, and communication protocols over standardized interfaces.

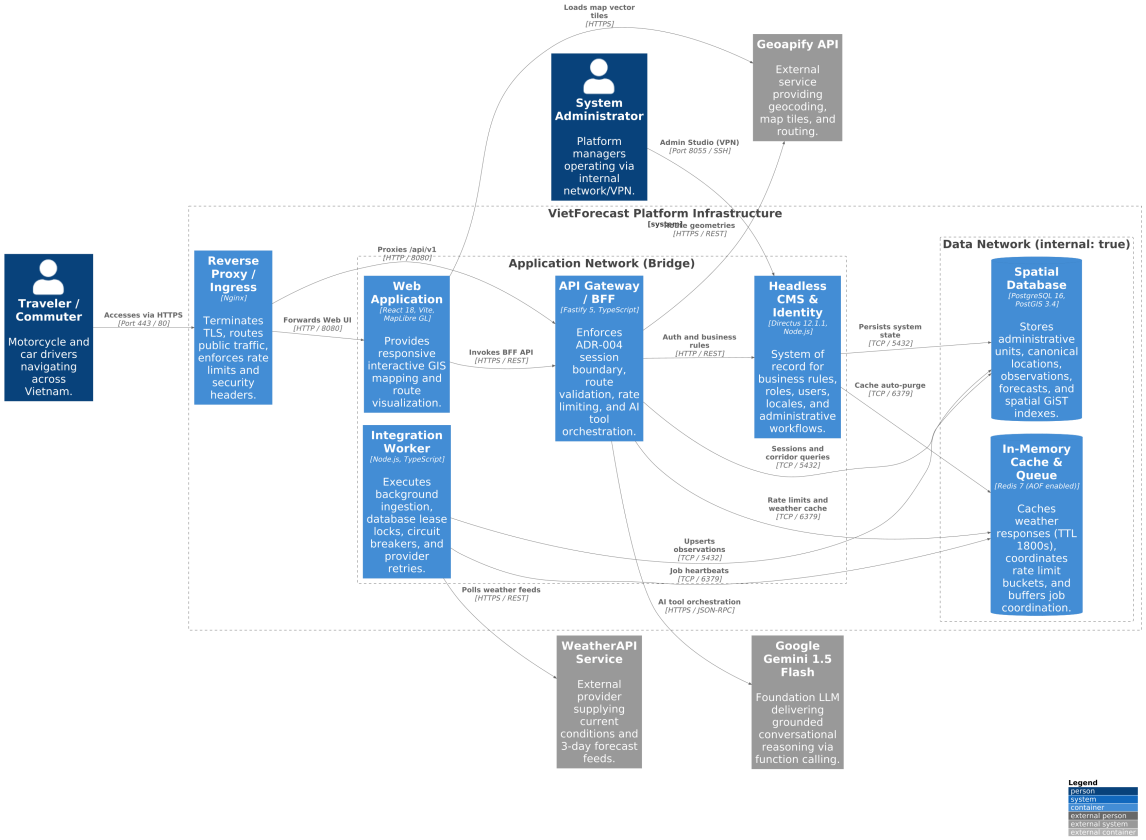


Figure 3.1: C4 Container Diagram (Level 2) illustrating the microservice topology and network segmentation of the VietForecast platform.

The main stack and responsibilities are:

- Nginx reverse proxy: public HTTPS entry point; terminates TLS, adds security headers, routes web/API traffic, and applies edge rate limits.
- React 18 + Vite + MapLibre GL 4 web app: renders weather layers, route lines, checkpoints, and account pages. It has no provider credentials or database connection.
- Fastify 5 + TypeScript API/BFF: serves typed endpoints, validates sessions, applies Redis limits, runs route-risk logic, and mediates assistant tools.
- Directus 12.1.1: manages users, RBAC, administrative data, risk rules, and localized content.
- Node.js Integration Worker: polls WeatherAPI, normalizes observations, computes checksums, and coordinates ingestion leases.
- PostgreSQL 16 + PostGIS 3.4: stores relational and spatial data; GiST indexes support corridor and nearby-station queries.
- Redis 7: provides short-lived weather caching, rate-limit tokens, and coordination signals.

3.1.2 Network Segmentation and Zero-Trust Isolation Policies (Docker Segmentation)

Docker Compose separates services into two networks to limit lateral movement:

- Application network (`application`): connects Nginx, the web app, Fastify, Directus, and the worker through Docker DNS and internal HTTP.
- Data network (`data`, `internal: true`): isolates PostgreSQL (5432) and Redis (6379) from host and public interfaces.

Directus Admin Studio is bound to `127.0.0.1:8055` (ADR-002, OPS-010). Administrators access it through an authenticated SSH tunnel or internal VPN.

3.2 Secure Session Boundary and Identity Architecture (ADR-004)

3.2.1 Token Exfiltration Vulnerability Analysis and BFF Boundary Architecture

Storing JSON Web Tokens in browser storage exposes them to XSS and couples the frontend to Directus schemas. VietForecast therefore keeps Directus communication inside the backend.

Under ADR-004, Fastify is the authenticated BFF. Directus is reached only on the private network, and browsers receive only an opaque session cookie.

3.2.2 AES-256-GCM Authenticated Encryption Envelope and Opaque Session Handles

The session model uses three controls:

- *Opaque Session Handle*: The Fastify server generates a 32-byte (256 bits) cryptographically secure pseudorandom string utilizing a CSPRNG (`createOpaqueToken(32)`). This handle is dispatched to the client browser exclusively within a hardened cookie named `__Host-vnwa_session`, enforcing four foundational security directives: `HttpOnly` (prohibits JavaScript access via `document.cookie`), `Secure` (restricts transmission strictly over encrypted TLS/HTTPS), `SameSite=Strict` (prevents CSRF in cross-origin requests), and prefix `__Host-` (binds the cookie immutably to the origin domain).
- *One-Way Token Hashing Pattern*: To safeguard active sessions in the event of an unauthorized database dump, the server never persists opaque handles in plaintext. Instead, the handle is passed through a SHA-256 cryptographic hash function before database insertion:

$$H_{\text{session}} = \text{SHA-256}(\text{handle}) \tag{3.1}$$

  The resulting 64-character hexadecimal digest H_{session} serves as the primary key (`session_id_hash`) in `app_security.sessions`. Even with full database access, an adversary cannot invert the hash to deduce valid client cookies.

- *AES-256-GCM Authenticated Encryption Envelope*: Upstream Directus tokens are encrypted at the Fastify tier employing Advanced Encryption Standard in Galois/Counter Mode (AES-256-GCM) with a 256-bit symmetric server key $K_{\text{enc}} \in \{0,1\}^{256}$:

$$\mathcal{C} = \text{AES-GCM-Enc}_{K_{\text{enc}}}(P, \text{IV}), \quad \text{Tag} = \text{GCM-Mac}_{K_{\text{enc}}}(\mathcal{C}, \text{IV}) \tag{3.2}$$

  Every session creation and rotation event generates an independent, random 12-byte initialization vector $\text{IV} \in \{0,1\}^{96}$ and yields a 16-byte authentication tag $\text{Tag} \in \{0,1\}^{128}$. The serialization envelope follows the standard structure:

$$\text{Envelope} = \texttt{v1} \parallel \text{base64url}(\text{IV}) \parallel \text{base64url}(\text{Tag}) \parallel \text{base64url}(\mathcal{C}) \tag{3.3}$$

  Encrypted payloads and authentication tags are persisted within the dedicated PostgreSQL table `app_security.sessions`. Table permissions are restricted exclusively to the Fastify service role (`API_DB_USER`), precluding even background worker processes from inspecting session states.

3.2.3 Unified Session Lifecycle: Issuance, Sliding Rotation, and Synchronized Revocation

Sessions use three stages: issuance, sliding rotation during activity, and revocation.

Phase 1: User Authentication and Opaque Session Handle Issuance

Figure 3.2 delineates the sequence execution of user authentication, RBAC authorization assertion, concurrency boundary locking, server-side encryption, and secure cookie issuance.

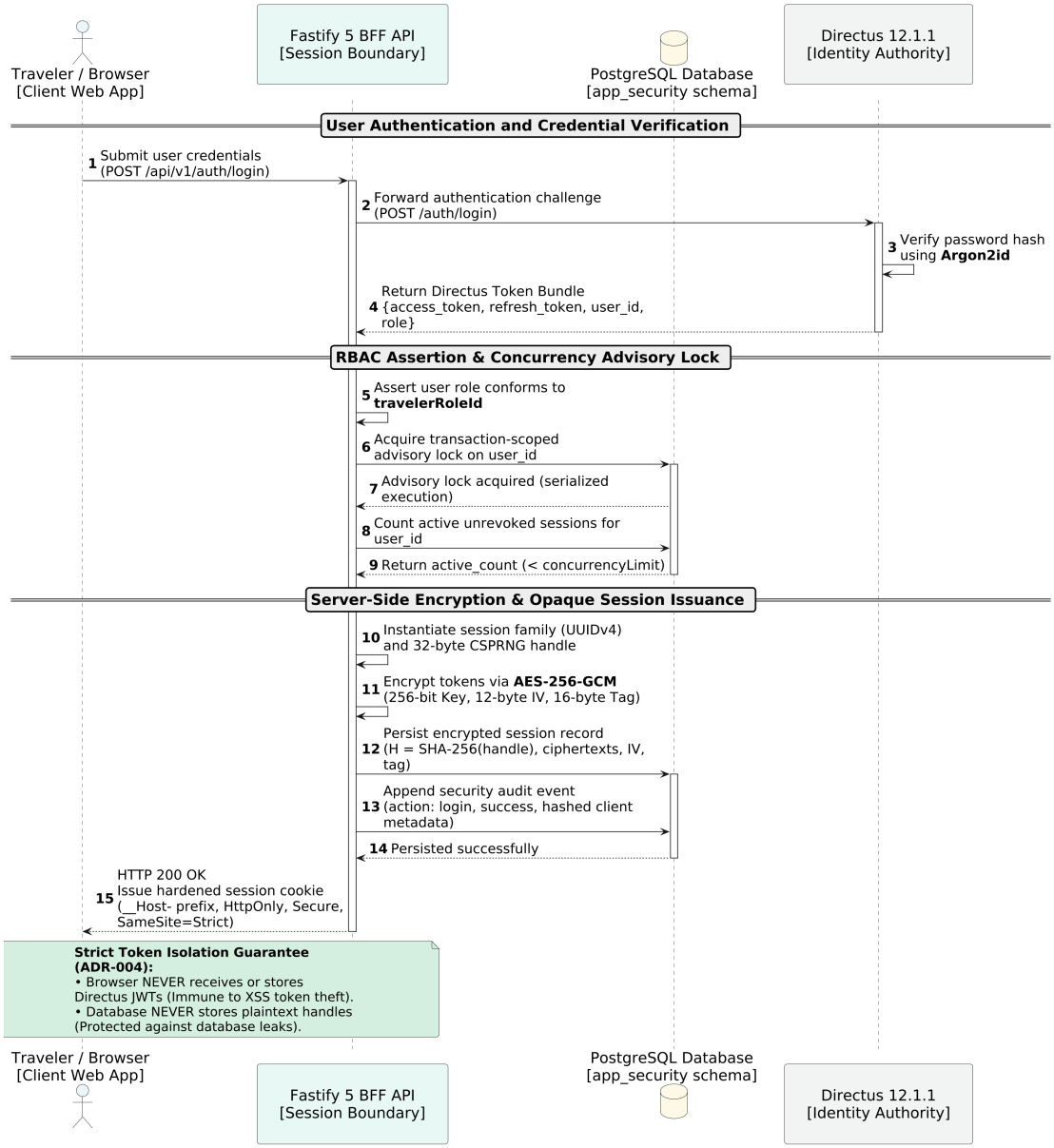


Figure 3.2: Sequence diagram of the secure session lifecycle (ADR-004) – Phase 1: User credential authentication, RBAC role assertion, concurrent session bounding, and opaque handle issuance.

The main steps are:

- The React/Vite client sends credentials to Fastify at `POST /api/v1/auth/login`; the API applies the login rate limit.
- Fastify asks Directus to verify the Argon2id password and receives the token bundle, user UUID, and role claims.

- Fastify checks the role and active-session limit. If the limit is reached, the oldest session is revoked:

$$N_{\text{active}}(u) = \left| \{ s \in \mathcal{S} \mid s.\text{user_id} = u \ \wedge \ s.\text{revoked_at} = \emptyset \} \right| < C_{\text{limit}} \tag{3.4}$$

- Fastify creates a 32-byte handle, encrypts Directus tokens with AES-256-GCM, hashes the handle with SHA-256, and stores the session and audit event in PostgreSQL.
- The API returns the `HttpOnly`, `Secure`, `SameSite=Strict`, `__Host-` session cookie.

Phase 2: Authorized Request Dispatch and Silent Sliding Rotation

Figure 3.2 details the sequence flow for handling authorized business queries, verifying session digests, and executing transparent sliding rotation.

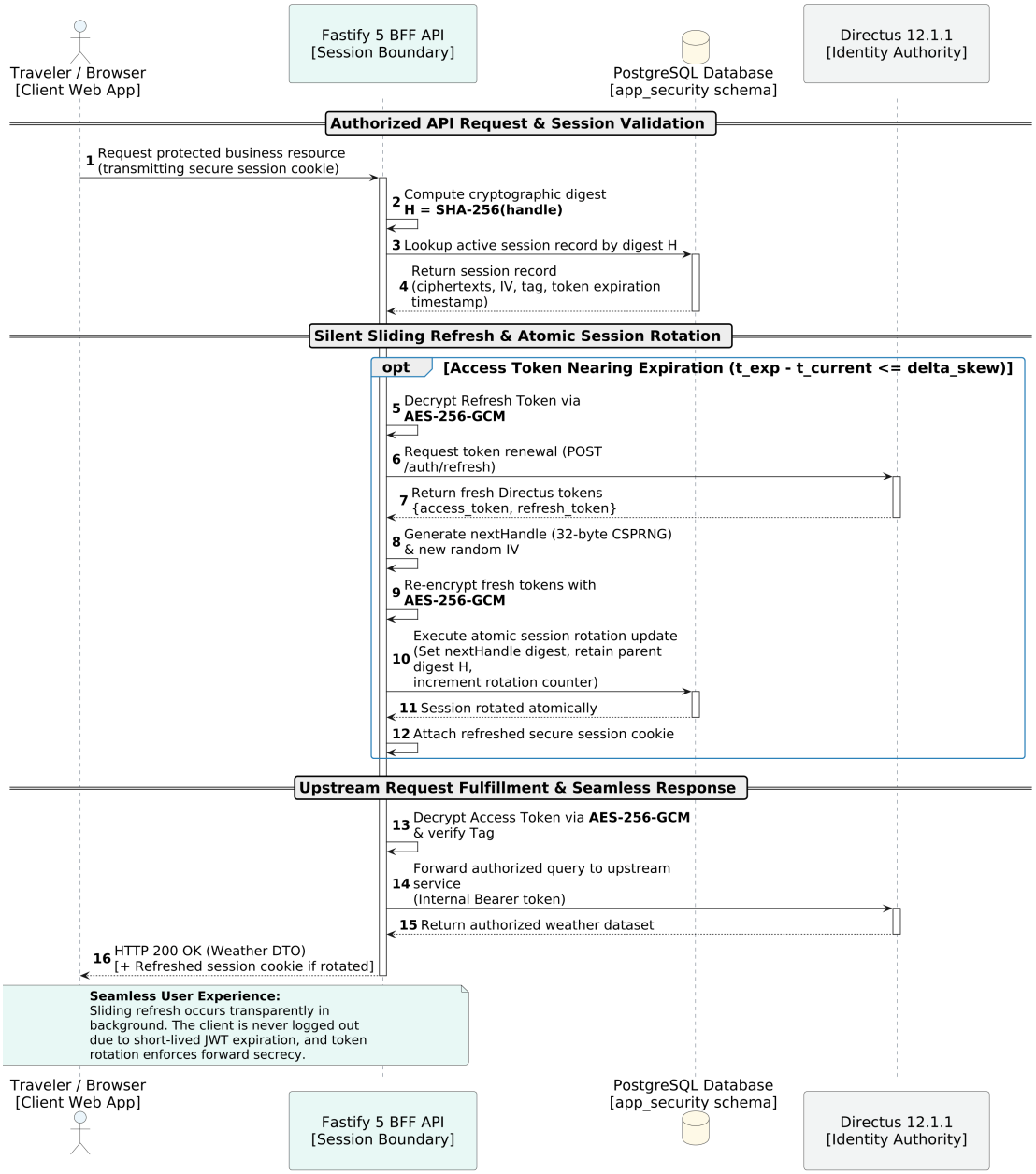


Figure 3.2: Sequence diagram of the secure session lifecycle (ADR-004) (cont.) – Phase 2: Authorized API request validation and silent sliding rotation (Silent Sliding Refresh).

The main steps are:

- Fastify hashes the session cookie and checks the active record in PostgreSQL or Redis. Invalid sessions receive HTTP 401.

- If the access token is near expiry, silent rotation starts when:

$$t_{\text{exp}} - t_{\text{current}} \leq \Delta t_{\text{skew}} \tag{3.5}$$

- Fastify refreshes the Directus tokens, creates a new handle, and atomically writes the next session state:

$$\begin{aligned} \mathcal{T}_{\text{rotate}}(S_k) \xrightarrow{\text{atomic}} S_{k+1} = \big\langle &\text{SHA-256}(h_{k+1}),\ \mathcal{C}_{k+1}, \\ &\text{IV}_{k+1},\ \text{Tag}_{k+1},\ t_{\text{exp},k+1},\ c_k + 1 \big\rangle \end{aligned} \tag{3.6}$$

  The previous hash remains linked for replay detection:

$$S_{k+1}.\text{ancestor_hash} = H_k \tag{3.7}$$

- Redis keeps the previous handle for 60 seconds so concurrent requests can finish. Fastify serves the request with the refreshed cookie.

Phase 3: Synchronized Revocation and Session Family Annihilation

Figure 3.2 depicts the sequence execution of synchronized event-driven trigger revocation and session family annihilation upon token reuse detection.

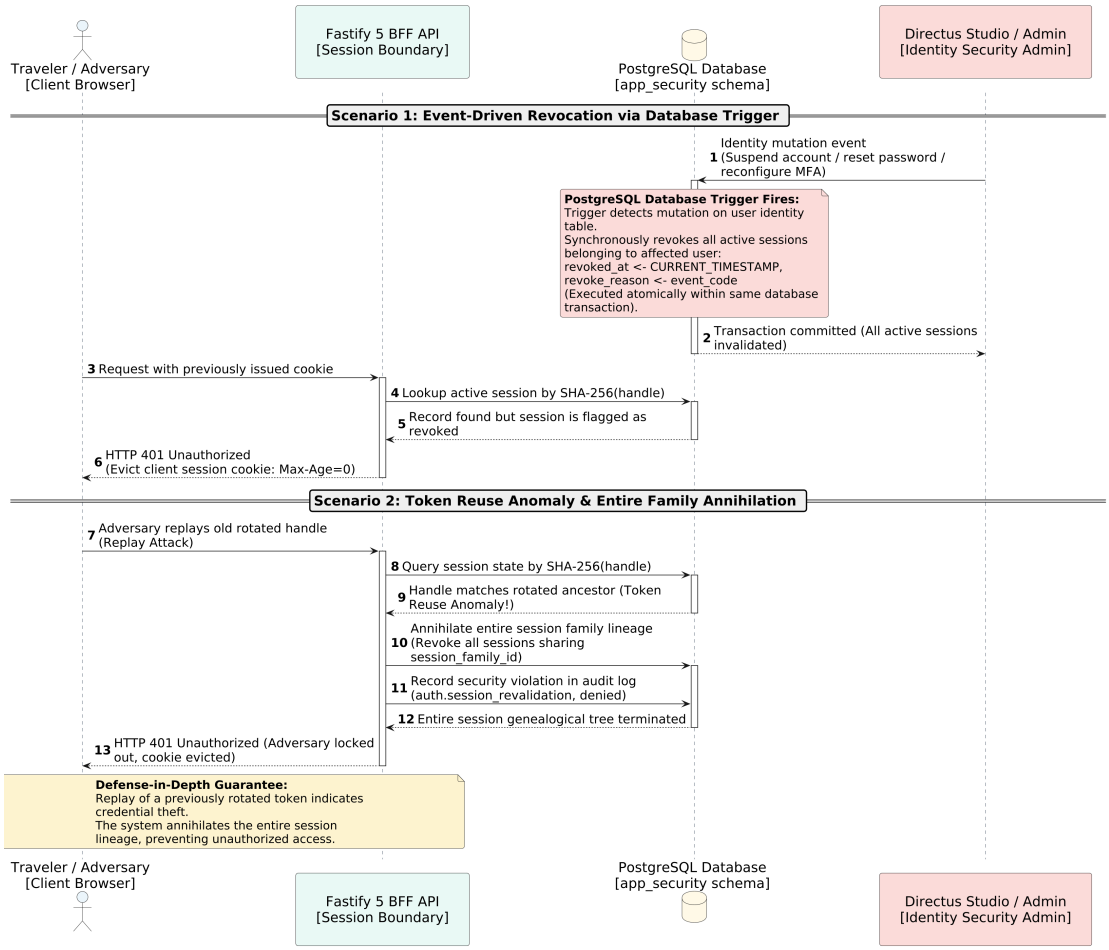


Figure 3.2: Sequence diagram of the secure session lifecycle (ADR-004) (cont.) – Phase 3: Synchronized database trigger revocation and session family annihilation upon token reuse detection.

The main controls are:

- Account, password, role, or MFA changes in Directus trigger PostgreSQL revocation for all active sessions of the user:

$$\forall s \in \mathcal{S},\ (s.\text{user_id} = u\ \wedge\ s.\text{revoked_at} = \emptyset)\ \Longrightarrow$$
$$\Phi_{\text{trigger}}(s) : \begin{cases} s.\text{revoked_at} \leftarrow t_{\text{event}} \\ s.\text{revoke_reason} \leftarrow \text{code}_{\text{reason}} \end{cases} \tag{3.8}$$

- Reuse of a rotated handle after the grace period marks the whole session family as revoked:

$$\forall s \in \mathcal{S},\ (s.\text{family_id} = F_{\text{target}})\ \Longrightarrow$$
$$\Psi_{\text{annihilate}}(s) : \begin{cases} s.\text{revoked_at} \leftarrow t_{\text{detection}} \\ s.\text{revoke_reason} \leftarrow \text{`token_reuse_detected'} \end{cases} \tag{3.9}$$

Redis descendants are removed and the event is recorded in the audit log.

- Logout revokes the PostgreSQL record, clears Redis and the cookie, and lets a retention task remove expired records.

3.3 Background Integration Worker and Resilience Patterns (ADR-001)

3.3.1 Fault Domain Isolation Between Background Tasks and APIs

WeatherAPI polling can fail or hit quotas, and collecting data for 63 provinces creates sustained network and normalization work.

ADR-001 moves ingestion, normalization, metric calculation, and cache updates to a separate Node.js Integration Worker, keeping the Fastify API responsive.

3.3.2 Distributed Database Leasing and Concurrency Control

The worker uses PostgreSQL table `weather_worker_leases` to coordinate multiple instances and prevent duplicate cycles.

The leasing mechanism enforces atomic acquisition with expiration:

- At the start of an ingestion cycle, a candidate worker attempts an atomic conditional upsert (`INSERT ... ON CONFLICT DO UPDATE`) on the lease lock record. Acquisition succeeds only if no worker currently holds the lease or the existing lease timestamp has lapsed.
- Lease tenure is configured by parameter `WEATHER_WORKER_LEASE_DURATION_MS` ($120,000$ ms).
- Throughout active data ingestion and normalization, the worker dispatches heartbeat pings every 30,000 milliseconds (`HEARTBEAT_INTERVAL_MS` ($30,000$ ms)) to extend lease validity.
- If a worker crashes or suffers network partition, heartbeat signals cease. Once the 120-second lease TTL lapses, the lease expires cleanly without distributed deadlock, allowing standby workers to assume ingestion duties seamlessly.

3.3.3 Circuit Breaker Pattern and Adaptive Backoff

The Integration Worker uses a three-state circuit breaker to protect provider quotas and local availability:

- *Closed State (CLOSED)*: WeatherAPI requests execute normally while consecutive failure counters are continuously tracked.

- *Open State (OPEN)*: When consecutive network failures (HTTP 5xx, HTTP 429, or timeouts exceeding `TIMEOUT_MS` ($8,000$ ms)) reach `CIRCUIT_FAILURE_THRESHOLD` (3 attempts), the circuit trips to Open. In this state, outgoing calls are immediately short-circuited locally without placing network load on upstream servers.
- *Half-Open State (HALF-OPEN)*: Upon expiry of `CIRCUIT_RESET_MS` ($60,000$ ms), the breaker transitions to Half-Open and issues a single probe request. If successful, the circuit resets to Closed; otherwise, it immediately re-enters Open and restarts the backoff timer.

For transient errors, the worker applies exponential backoff with full jitter. Delay duration for retry attempt k is calculated as:

$$t_{\text{delay}}(k) = \min\left(t_{\text{max}},\ t_{\text{base}} \cdot 2^{k-1}\right) \cdot \left(1 + r_{\text{jitter}} \cdot \mathcal{U}(-1, 1)\right) \tag{3.10}$$

with base backoff $t_{\text{base}} = 5,000$ ms, maximum backoff $t_{\text{max}} = 60,000$ ms, jitter ratio $r_{\text{jitter}} = 0.2$, and $\mathcal{U}(-1, 1)$ representing a continuous uniform random variable on $[-1, 1]$. This stochastic jitter prevents thundering herd contention across station retry cycles.

3.3.4 Meteorological Data Freshness Lifecycle and Safe States

The worker normalizes observations to WMO-No. 8 units, records a SHA-256 digest, updates `weather_observations` and `weather_forecasts`, and refreshes Redis keys.

Persisted weather observations are governed by a three-tier freshness state model to ensure the validity and safety of route hazard evaluations:

- *Fresh State*: Observation latency $\tau \leq 60$ minutes from station observation timestamp. Fully validated and representative of actual atmospheric conditions; ingested directly into the route risk scoring engine.
- *Aging State*: Observation latency $60 < \tau \leq 180$ minutes. Data remains consumable for corridor routing computations, but client interfaces display an aging timestamp advisory to inform travelers.
- *Stale State*: When observation latency $\tau > 180$ minutes (or weather station disconnection exceeds 3 hours). Under Fail-Safe engineering principles, stale telemetry is rejected for hazard derivation and automatically mapped to the `Unknown` risk tier (neutral gray #9CA3AF), guaranteeing users are alerted to missing field observations rather than receiving unverified safety assurances.

Figure 3.3 illustrates the end-to-end background ingestion lifecycle, covering distributed lease acquisition, circuit breaker evaluation, data normalization, and cache hydration.

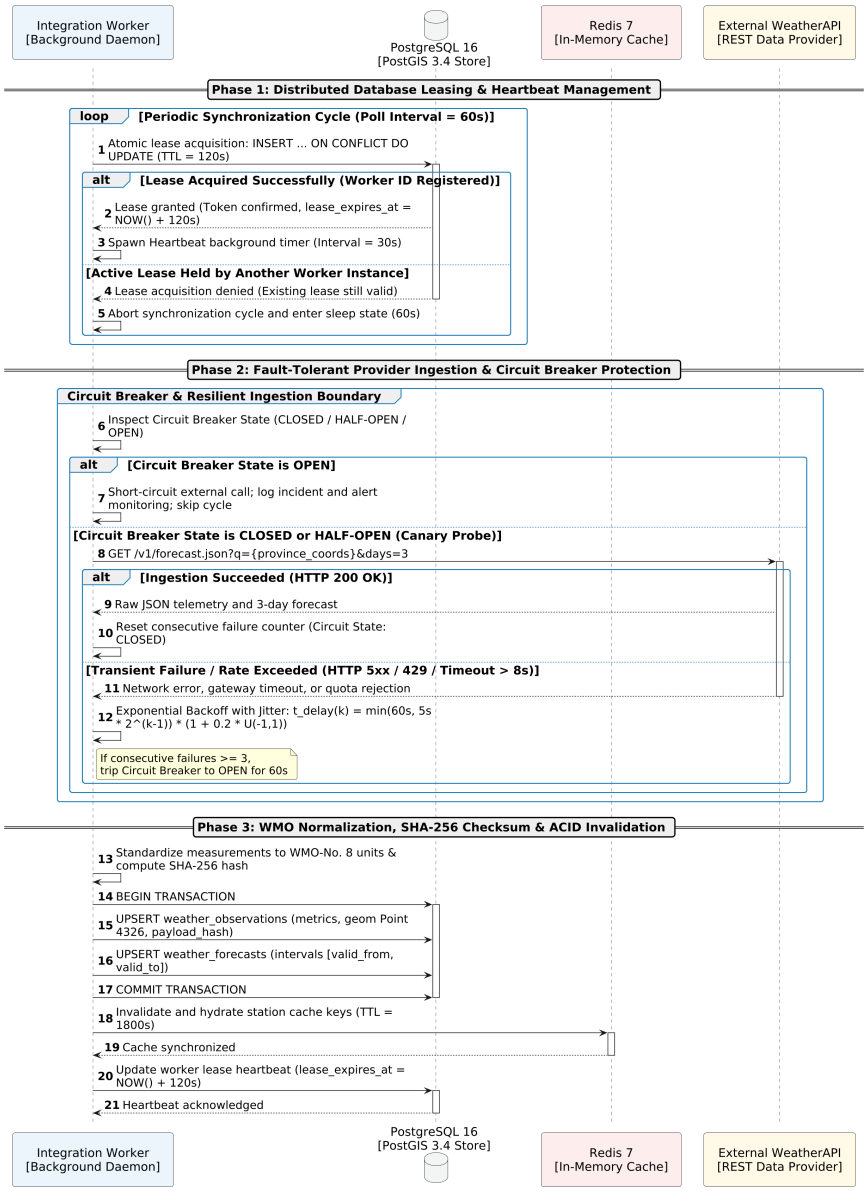


Figure 3.3: Sequence diagram of background meteorological ingestion, distributed leasing, and circuit breaker protection (ADR-001).

The worker executes three phases:

- Phase 1: Every 60 seconds, the worker acquires the PostgreSQL lease with an atomic upsert. The owner renews the 120-second lease every 30 seconds; other instances wait.
- Phase 2: The worker checks the circuit state before calling `GET /v1/forecast.json`. Success resets failures. HTTP 5xx, 429, or timeout errors use the backoff in Equation (3.10); three failures open the circuit for 60 seconds.
- Phase 3: The worker normalizes the response, computes a SHA-256 digest, upserts observations and forecasts in one transaction, then refreshes Redis (TTL 1800 seconds) and the lease heartbeat.

3.4 PostGIS Spatial Database Schema and Entity-Relationship Architecture

3.4.1 Tiered Database Domain Architecture

The VietForecast persistent store operates on PostgreSQL 16 enhanced with the PostGIS 3.4 spatial extension. The database schema is partitioned into modular functional domains:

- *Security and Session Domain* (`app_security`): Contains `app_security.sessions` storing AES-256-GCM encrypted token envelopes and `app_security.audit_events` capturing immutable audit logs.
- *Administrative and Spatial Domain* (`administrative`): Contains `administrative_units` detailing Vietnam's administrative tree (Country $\rightarrow$ 63 Provinces $\rightarrow$ Districts $\rightarrow$ Communes) with `MultiPolygon` boundaries, and `locations` cataloging observation stations, landmarks, and transportation nodes with `geometry(Point, 4326)`.
- *Meteorological Data Domain* (`weather`): Contains `weather_providers`, `weather_observations` storing surface time-series metrics with SHA-256 checksums, `weather_forecasts` holding interval validity ranges $[t_{\text{start}}, t_{\text{end}}]$, and `weather_worker_leases` coordinating distributed worker locks.
- *Risk Rule and Route Domain* (`risk_route`): Houses `risk_rule_sets` and `risk_rules` under Maker-Checker governance, `risk_assessments` with audit evidence, alongside `route_requests`, `route_options`, and `route_samples` holding LineString geometries and spatiotemporal sample waypoints.

3.4.2 Spatial Query Optimization with GiST Indexes

In highway corridor hazard evaluation, the platform must query all meteorological observation stations situated within an $r_{\text{corridor}} = 15$ km buffer corridor surrounding a travel itinerary spanning hundreds of kilometers. Evaluating geodetic distance across entire tables via sequential scans incurs $O(M \cdot N)$ complexity (where M is the route sample count and N is total observation records), causing latency spikes of several seconds.

PostGIS 3.4 overcomes this limitation by implementing Generalized Search Tree (GiST) indexing over all geometry columns. GiST structures spatial features as hierarchical bounding box trees (Minimum Bounding Box – BBOX). The spatial engine executes a two-tier filtering strategy:

- *Rough Pass (Bounding Box Filter)*: The query engine evaluates intersection between the bounding box of the route buffer corridor $G_{\text{corridor}} = \text{ST_Buffer}(\mathcal{L}, r_{\text{corridor}})$ and candidate station bounding boxes:

$$\text{BBOX}(o.\text{geom}) \cap \text{BBOX}(G_{\text{corridor}}) \neq \emptyset \tag{3.11}$$

  Traversing the balanced GiST index requires only $O(\log N)$ operations, filtering out over 95% of distant stations without expensive spherical trigonometry.

- *Exact Pass*: PostGIS applies exact ellipsoidal geodesic distance computations (`ST_DWithin` on `geography` data types) exclusively to candidates surviving the rough pass:

$$d_{\text{geodetic}}(o.\text{geom}, \mathcal{L}) \leq r_{\text{corridor}} \quad \text{with } \tau(o) \leq 180 \text{ minutes} \tag{3.12}$$

ensuring mathematical precision under the WGS84 ellipsoid in under 15 milliseconds. The concrete spatial GiST indexing parameters and two-tier corridor query specifications are detailed in Table 4.1 in Chapter 4.

3.4.3 Standardized Entity-Relationship Diagram and Governance Constraints

Figure 3.4 depicts the Entity-Relationship Diagram (ERD) of the VietForecast core data architecture, specifying primary keys, foreign key linkages, and referential constraints.

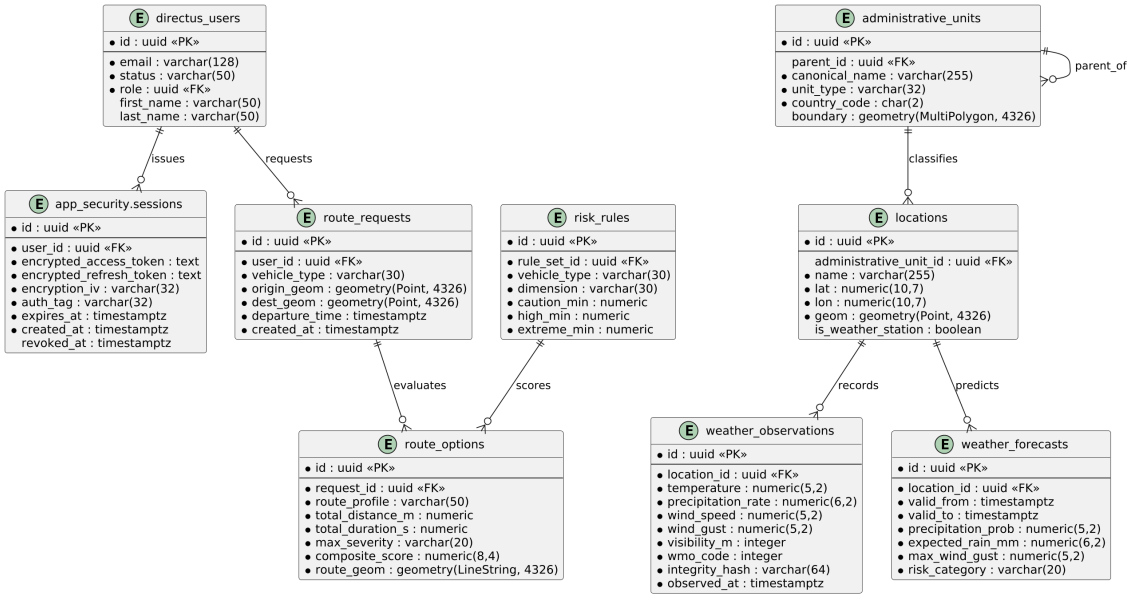


Figure 3.4: Entity-Relationship Diagram (ERD) illustrating spatial data structures, identity security, and risk rule governance in VietForecast.

The `directus_users` entity serves as the authoritative identity anchor across the platform, binding active security sessions in `app_security.sessions` to historical routing queries in `route_requests`. Two advanced governance mechanisms enforce structural integrity:

- *Maker-Checker Governance for Risk Rules*: Risk rule sets within `risk_rule_sets` transition sequentially across: `draft`, `review`, `approved`, `published`, and `retired`. Database trigger `app_governance.guard_risk_rule_mutation()` prohibits modifications or deletions of child rules in `risk_rules` once a set departs `draft`, ensuring historical hazard judgments remain immutable and deterministic.

- *Immutable Security Audit Trail*: All administrative mutations regarding credentials, access roles, or threshold parameters automatically emit records into `app_security.audit_events` via `SECURITY DEFINER` triggers, preventing unauthorized tampering with security history.

3.5 Multi-Objective Route Risk Evaluation and Balancing Engine

3.5.1 Spatiotemporal Modeling and Route Corridor Discretization

The route-risk engine converts sampled weather evidence into guidance for drivers.

The platform queries route geometries from the road routing engine (OSRM), acquiring three alternative itineraries: Fastest, Shortest, and Alternative, each defined as a GeoJSON LineString accompanied by total distance D and nominal travel duration T. The engine discretizes each polyline into a sequence of sample waypoints $p_i = (\phi_i, \lambda_i)$ under dual spatial and temporal constraints: inter-sample spatial separation $\Delta s \leq 5$ km and temporal advance $\Delta t \leq 15$ minutes, computed from the nominal velocity $\bar{v} = D/T$.

The Fastify API queries PostGIS using `ST_DWithin` to aggregate weather stations and forecast grids within a 15 km corridor around each waypoint p_i, interpolating atmospheric conditions at expected arrival time t_i.

3.5.2 Vehicle-Differentiated Hazard Threshold Matrix

The risk engine cross-references meteorological parameters at each waypoint p_i against a six-dimensional hazard ruleset (R_{rain}, R_{wind}, R_{temp}, R_{vis}, R_{storm}, R_{alert}). Waypoint hazard severity $S(p_i)$ follows the Max-Severity Aggregation principle:

$$S(p_i) = \max_{d \in \mathcal{D}} R_d(p_i, v) \tag{3.13}$$

where $\mathcal{D} = \{\text{rain}, \text{wind}, \text{temp}, \text{vis}, \text{storm}, \text{alert}\}$ denotes the dimension set, $v \in \{\text{motorcycle}, \text{car}\}$ denotes vehicle type, and $R_d \in \{0, 1, 2, 3\}$ corresponds to `Unknown` (0), `Low` (1), `Caution` (2), and `High` (3).

Table 3.1 delineates the vehicle-differentiated activation thresholds calibrated to Vietnamese road transport safety norms.

Table 3.1: Vehicle-differentiated weather hazard activation matrix (RISK-001, RISK-002).

Hazard Dimension	Metric & Unit	Caution Threshold	High Hazard Threshold
Precipitation (R_{rain})	Rainfall rate (mm/h)	Motorcycle: $10 \leq r < 25$ Car: $15 \leq r < 35$	Motorcycle: $r \geq 25$ Car: $r \geq 35$
Wind gust (R_{wind})	Wind gust speed (km/h)	Motorcycle: $25 \leq w < 40$ Car: $45 \leq w < 60$	Motorcycle: $w \geq 40$ Car: $w \geq 60$
Visibility (R_{vis})	Horizontal visibility (m)	Motorcycle: $1,000 \leq v < 2,000$ Car: $500 \leq v < 1,000$	Motorcycle: $v < 1,000$ Car: $v < 500$
Temperature (R_{temp})	Surface temperature (°C)	Motorcycle: 36–38°C hoặc 10–13°C Car: 38–40°C hoặc 5–8°C	Motorcycle: > 38°C hoặc < 10°C Car: > 40°C hoặc < 5°C
Thunderstorm (R_{storm})	WMO convective activity	Distant storm or cumulonimbus	Direct thunderstorm or hail
Severe Alert (R_{alert})	National disaster tier	Disaster Risk Level 1–2	Disaster Risk Level 3+

3.5.3 Balanced Multi-Objective Route Optimization Function

The system scores candidate routes utilizing a normalized multi-objective scalarization objective:

$$Score_{\text{balanced}} = w_{\text{risk}} \cdot \hat{S}_{\text{route}} + w_{\text{time}} \cdot \hat{T} + w_{\text{dist}} \cdot \hat{D} \tag{3.14}$$

with default weights $w_{\text{risk}} = 0.5$, $w_{\text{time}} = 0.3$, and $w_{\text{dist}} = 0.2$. Normalized route hazard $\hat{S}_{\text{route}} = \frac{1}{K}\sum_{k=1}^{K} S(p_k)/3$, distance $\hat{D} = D/\max(D)$, and travel duration $\hat{T} = T/\max(T)$ map to $[0, 1]$. The lowest-scoring corridor receives the Weather-Safer Recommended designation, balancing transit efficiency against severe weather exposure.

3.5.4 Route Hazard Evaluation Pipeline and Sequence Diagram

Figure 3.5 details the comprehensive sequence flow of multi-alternative hazard evaluation upon user route submission via the MapLibre GL frontend.

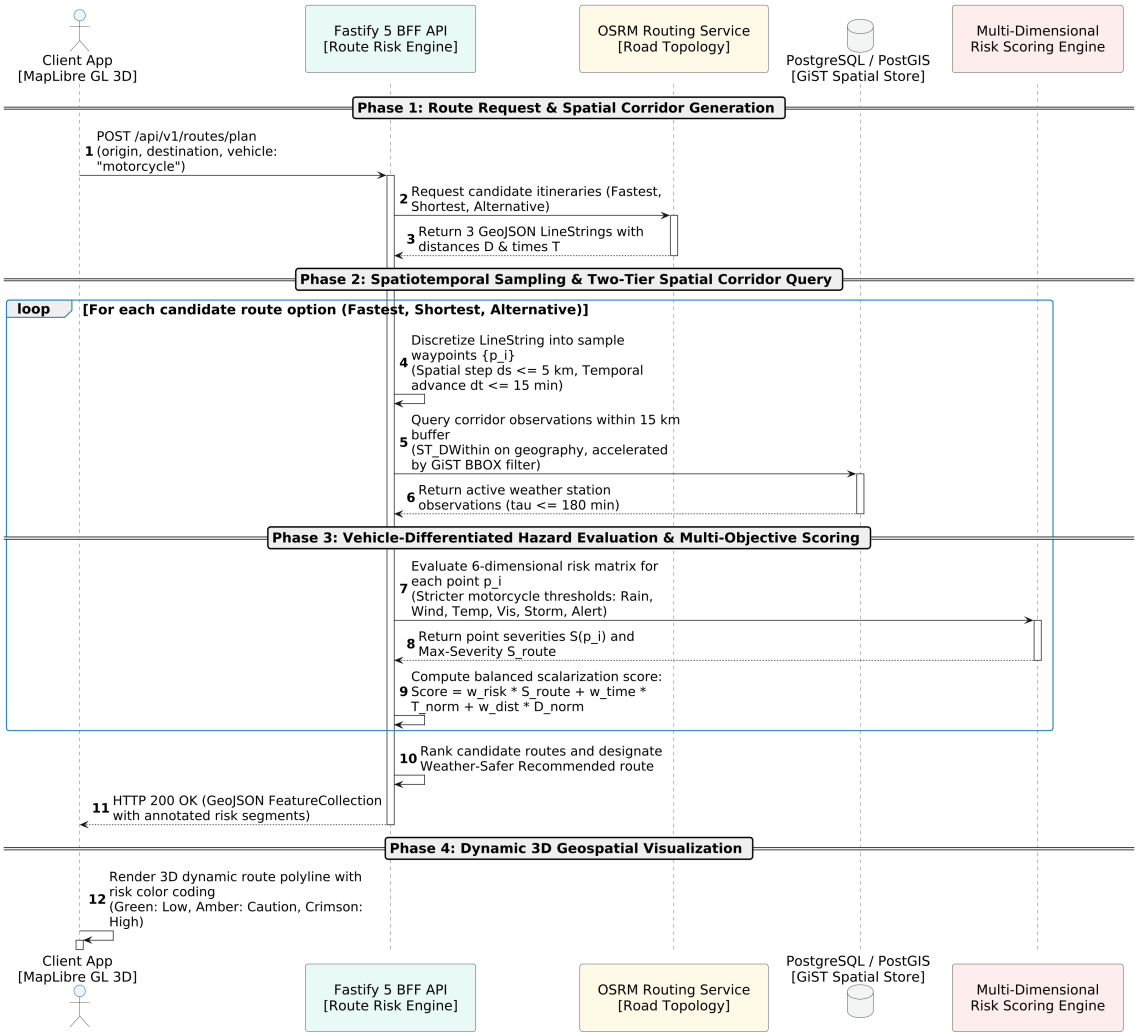


Figure 3.5: Sequence diagram of multi-objective route risk evaluation, spatiotemporal sampling, and 3D map visualization.

The route pipeline has four phases:

- *Phase 1: Request and routing*: The React app sends `POST /api/v1/routes/plan` to Fastify. OSRM returns Fastest, Shortest, and Alternative GeoJSON routes with distance D and duration T.
- *Phase 2: Sampling and spatial lookup*: Fastify samples each LineString at up to 5 km or 15-minute intervals and queries PostGIS `ST_DWithin` within a 15 km corridor. GiST filtering selects weather evidence with freshness $\tau \leq 180$ minutes.
- *Phase 3: Risk and scoring*: The six hazard dimensions are evaluated with the vehicle-specific rules. The engine applies Max-Severity Aggregation and Equation (3.14); the lowest score receives Weather-Safer Recommended.
- *Phase 4: Map response*: Fastify returns a GeoJSON FeatureCollection. MapLibre GL 4 renders route segments and Low, Caution, and High colors on the interactive map.

3.6 Grounded Conversational Weather AI Engine

3.6.1 Tool Orchestration Architecture

The optional Grounded AI Engine uses Google Gemini 1.5 Flash with backend tool execution.

Gemini parses intent and selects one of five backend tools. It does not provide weather values without returned evidence:

- `search_location`: Queries spatial coordinates, administrative divisions, and points of interest across Vietnam.
- `get_current_weather`: Retrieves validated real-time surface observation telemetry from monitoring stations.
- `get_forecast`: Fetches time-series forecast projections for specified intervals.
- `assess_travel_risk`: Computes localized point hazard classifications according to vehicle profiles.
- `evaluate_route`: Assesses route safety corridors via mapping function:

$$\mathcal{F}_{\text{route}} : (\text{origin}, \text{destination}, \text{vehicleType}) \longrightarrow \text{RouteRiskAssessment} \tag{3.15}$$

  The formal parameter specification for the `evaluate_route` tool is documented in Table 4.2 in Chapter 4.

3.6.2 Grounded Evidence Pipeline and Citation Synthesis

The evidence flow has three steps:

- *Tool invocation*: Gemini selects a tool. Fastify executes the PostGIS and routing query, checks freshness $\tau \leq 180$ minutes, and returns normalized evidence.
- *Response*: Gemini summarizes only the returned data and includes station identifiers, timestamps, and safety advice.
- *Fallback*: If Gemini is unavailable, Fastify's `GroundedRuleEngineAdapter` uses location matching and deterministic rules to keep weather and risk responses available.

Chapter 4

System Implementation

This chapter maps the architecture to the implementation. It focuses on the TypeScript monorepo, Docker stack, Fastify API, secure sessions, PostGIS queries, weather worker, route-risk engine, assistant, React/MapLibre frontend, and testing pipeline.

4.1 Development Environment and Deployment Stack

4.1.1 Monorepo Architecture and Static Type Contracts

VietForecast uses a `pnpm workspaces` monorepo with TypeScript 5.5 strict mode and `NodeNext` modules.

The repository has two workspace groups:

- *Application workspace (`apps/`)*: Houses independently deployable processes, including the Fastify API / BFF gateway (`apps/api`), the background meteorological ingestion worker (`apps/worker`), the headless Directus 12.1.1 IAM and CMS service (`apps/directus`), and the React 18 single-page application (`apps/web`).
- *Shared library workspace (`packages/`)*: Houses reusable infrastructural modules, including `packages/contracts` (defining shared Data Transfer Objects (DTOs), Zod/Ajv schema validators, and canonical domain constants) and `packages/config` (standardizing TypeScript compiler configurations and ESLint 9 linting rules).

Shared DTOs, Zod/Ajv schemas, and constants in `packages/contracts` keep frontend and back-end payloads consistent.

4.1.2 Three-Tier Network Partitioning and Containerization

Docker Compose packages the services into OCI containers and separates them into three networks:

- *Edge and Reverse Proxy Tier (`vnwa_edge_net`)*: Directly exposes ports 80 (HTTP) and 443 (HTTPS) to the public Internet through an Nginx reverse proxy. Nginx terminates TLS sessions, injects strict HTTP security headers (CSP, HSTS, X-Frame-Options), and routes ingress traffic to internal web and API ports.

- *Application and Orchestration Tier* (`vnwa_app_net`): Hosts `apps/api`, `apps/worker`, and `apps/directus` containers. These services communicate with one another, query persistent datastores, and dispatch outbound requests to external weather telemetries and foundational AI endpoints.

- *Isolated Datastore Tier (`vnwa_data_net`)*: Houses PostgreSQL 16 (equipped with PostGIS 3.4) and the Redis 7 in-memory cache. Crucially, this network is flagged with `internal: true`, which severs the default Internet gateway. Consequently, datastore containers cannot be accessed directly from external networks, nor can compromised database processes transmit unauthorized outbound payloads across the Internet.

Health checks use `pg_isready`, `redis-cli ping`, and API/worker `/health` or `/ready` endpoints.

4.2 Backend Service Layer and Session Security Boundary Implementation (ADR-004)

4.2.1 Fastify 5 Request Lifecycle and Hook Pipeline

The `apps/api` service uses Fastify 5 and typed lifecycle hooks. Each request passes through:

- *Ingress stage (`onRequest`)*: Invokes the Redis 7 distributed Token Bucket rate limiter. By inspecting the client IP address and hashed session handle, Fastify rejects traffic exceeding preconfigured burst thresholds with HTTP 429 (*Too Many Requests*) and standard `Retry-After` headers without consuming downstream computational resources.

- *Validation stage (`preValidation`)*: Compiles incoming request bodies, route parameters, and query strings against shared JSON Schemas from `packages/contracts` via Ajv. Malformed or poisoned payloads are rejected with HTTP 400 (*Bad Request*).

- *Authentication stage (`preHandler`)*: Extracts the opaque session handle from the secure `__Host-vnwa_session` cookie. The hook hashes the handle via SHA-256, verifies its validity against `app_security.sessions`, executes silent sliding rotation when the underlying token approaches expiration, and injects authenticated user claims (`userId`, `roleId`) into the request context.

- *Telemetry and audit stage (`onResponse`)*: Records microsecond-level request durations for Prometheus metric scraping and synchronously dispatches security audit records to `app_security.audit_events` upon sensitive administrative mutations.

4.2.2 Atmospheric Cryptographic Envelope and Timing-Safe Protections

Cryptographic routines are centralized in `apps/api/src/crypto.ts` using native `node:crypto`:

- *AES-256-GCM authenticated envelope*: Upstream Directus access and refresh tokens are encrypted at rest using Galois/Counter Mode. Function `encryptSecret` extracts 96 bits

(12 bytes) of cryptographic entropy via CSPRNG (`randomBytes(12)`) as the Initialization Vector (IV), encrypts the secret using the 256-bit key `API_SESSION_ENCRYPTION_KEY`, and captures the 128-bit authentication tag (`authTag`). The payload is serialized as:

$$\text{Envelope} = \text{"v1"} \,.\, \text{Base64URL}(\text{IV}) \,.\, \text{Base64URL}(\text{Tag}) \,.\, \text{Base64URL}(\text{Ciphertext}) \quad (4.1)$$

Function `decryptSecret` enforces the `v1` protocol version and validates the authentication tag; any bit-level tampering aborts execution immediately with an unhandled exception.

- *CSPRNG opaque token generation and one-way hashing*: The client-facing handle is synthesized by `createOpaqueToken(32)`, drawing 32 bytes of entropy from CSPRNG and encoding it into a 43-character URL-safe string (`base64url`). Before persistence, the handle is digested via SHA-256 (`hashOpaqueToken`) and written to `session_id_hash`. This one-way transformation guarantees that database read compromises yield no usable bearer tokens.

- *Timing-safe CSRF validation*: CSRF prevention relies on HMAC-SHA256 signatures (`signCsrfToken`). Signature verification in `verifyCsrfSignature` utilizes `timingSafeEqual`, ensuring execution time remains constant regardless of string prefix similarity, thus neutralizing timing side-channel cryptanalysis.

4.2.3 Durable Session Store and Session Family Annihilation (ADR-004)

`PostgresSessionStore` in `apps/api/src/session-store.ts` implements ADR-004:

- *Concurrency limits via transaction-level advisory locks*: To eliminate race conditions where parallel login challenges exceed the account concurrency ceiling C_{limit}, method `create` executes inside an explicit PostgreSQL transaction and acquires an exclusive advisory lock:

$$\text{SELECT pg_advisory_xact_lock}(\text{hashtextextended}(u.\text{user_id}, 0)) \quad (4.2)$$

  This serializes concurrent logins for that specific user ID without blocking other platform tenants. Active sessions (`revoked_at IS NULL` and `expires_at > now()`) are tallied; if the count reaches C_{limit}, the transaction aborts with an `ApiError(409, "SESSION_LIMIT_REACHED")`.

- *Atomic silent sliding rotation*: Rotation executes via a single atomic SQL `UPDATE` on `app_security.sessions`, storing the prior token hash into `rotated_from_hash`, incrementing `rotation_counter`, and advancing `expires_at`. A 60-second grace window is granted to the retired hash, enabling concurrent in-flight browser requests across multiple tabs to complete cleanly.

- *Token replay detection and session family annihilation ($\Psi_{annihilate}$)*: If an inbound request presents a token whose hash matches `rotated_from_hash` after the grace period has elapsed, the store identifies an active token hijacking attempt. Method `revokeFamily` executes instant cascade invalidation:

$$\begin{aligned} &\text{UPDATE app_security.sessions} \\ &\quad \text{SET revoked_at} = t_{\text{now}},\ \text{revoke_reason} = \text{"TOKEN_REUSE_DETECTED"} \\ &\quad \text{WHERE session_family_id} = \mathcal{F}_{\text{id}} \end{aligned} \quad (4.3)$$

All active sessions tied to the compromised family are invalidated simultaneously, ejecting the intruder and requiring legitimate users to re-authenticate.

4.3 PostGIS Spatial Database and GiST Query Optimization

4.3.1 Spatial Schema Architecture and GiST Index Declarations

PostgreSQL 16 with PostGIS 3.4 stores the spatial model, including:

- *Administrative catalog and stations (`locations`)*: Encapsulates spatial coordinates `geom` typed as `GEOMETRY(Point, 4326)` representing Vietnam's 63 provinces and fixed weather stations, accompanied by polygonal boundaries `boundary` typed as `GEOMETRY(MultiPolygon, 4326)` under the WGS84 standard.
- *Meteorological observations (`weather_observations`)*: Stores surface atmospheric observations, linked to `locations` via foreign key `location_id`. Spatial column `geom` (`GEOMETRY(Point, 4326)`) is duplicated on this table to optimize spatial query locality during corridor scans.

GiST indexes are created on both geometry columns:

```
CREATE INDEX idx_weather_obs_geom ON weather_observations USING GIST (geom)
```
(4.4)

```
CREATE INDEX idx_locations_geom ON locations USING GIST (geom)
```
(4.5)

GiST organizes geometries as bounding boxes and lets PostgreSQL find candidates in $O(\log N)$ time.

4.3.2 Two-Tier Spatial Corridor Query Specification

The corridor query uses a 15 km buffer, two spatial filters, and a freshness check.

Table 4.1 provides the formal technical specification of the corridor query parameters, predicates, and performance metrics.

Table 4.1: Technical specifications and predicate filters for the PostGIS 3.4 spatial corridor query.

Query Aspect	Technical Specification	Operational Function and Purpose
Input Geometry	Route Polyline $\mathcal{L}$ Type: `LineString` WGS84	Sequence of coordinates (ϕ_k, λ_k) representing the candidate travel corridor computed by OSRM.
Corridor Radius	$r_{\text{corridor}} = 15,000$ meters	Lateral meteorological influence perimeter flanking the travel polyline.
Rough Pass Predicate	$\text{BBOX}(o.\text{geom}) \cap$ $\text{BBOX}(\mathcal{L}) \neq \emptyset$ Complexity: $O(\log N)$	Navigates the 2D GiST index tree to instantly prune stations whose bounding boxes do not intersect the trajectory; eliminates over 95% of candidate search space.
Exact Pass Predicate	`ST_DWithin`(o.geom :: geography, $\mathcal{L}$ :: geography, 15000)	Casts geometries to `geography` and evaluates true ellipsoidal geodesic distance on the WGS84 datum.
Freshness Filter	o.observed_at $\geq$ $t_{\text{now}} - 180$ min ($\tau(o) \leq 180$ min)	Discards outdated telemetry; admits exclusively *Fresh* or *Aging* observation records.
Observed Latency	Latency: < 15 ms Plan: *Bitmap Index Scan*	Leverages PostgreSQL shared memory page caching; satisfies real-time route computation deadlines.

`ST_DWithin` on `geography` provides exact WGS84 distance after the GiST pre-filter. The target query latency is under 15 ms.

4.4 Background Integration Worker and Resilience Engine Implementation (ADR-001)

4.4.1 Distributed PostgreSQL Database Leasing Mechanism

The Node.js service `apps/worker` polls weather data outside client requests. ADR-001 uses `weather_worker_leases` to coordinate replicas.

Leader leasing is coordinated through two specialized PostgreSQL stored functions:

- *Run acquisition (`claim_weather_run`)*: Workers execute an atomic `INSERT ... ON CONFLICT (provider_id, target_location_id) DO UPDATE` statement using row-level locks (`FOR UPDATE SKIP LOCKED`). This skips rows actively locked by peer replicas without thread contention. Leases are granted with a fixed tenure of $T_{\text{lease}} = 120$ seconds.
- *Heartbeat renewal (`heartbeat_weather_run`)*: During long-running network fetches, workers transmit heartbeat signals at 30-second intervals to advance `lease_expires_at`. If an active worker crashes, its lease expires naturally after 120 seconds, allowing standby workers to recover pending workloads automatically without human intervention.

4.4.2 Three-State Circuit Breaker and Exponential Backoff with Jitter

`WeatherDurableRuntime` in the worker implements:

- *Three-state Circuit Breaker*: Protects workers against upstream provider outages or quota exhaustion from WeatherAPI. The state machine transitions across three operational modes:
  + *Closed*: Normal operation. Ingestion requests are dispatched directly to WeatherAPI. Recording $F \geq 3$ consecutive failures or quota depletion errors (`weather_provider_quota_exhausted`) triggers an immediate transition to *Open*.
  + *Open*: Circuit trips. Inbound poll attempts fail immediately in memory without network dispatch, preserving worker threads. The open state persists across cooldown interval $T_{\text{cooldown}} = 60$ seconds.
  + *Half-Open*: Upon cooldown expiration, the circuit permits a single canary probe request. If successful, the failure counter resets to zero and the circuit returns to *Closed*; if the probe fails, the circuit re-opens and doubles the cooldown duration.
- *Exponential Backoff with Stochastic Jitter*: Upon transient network interruptions, retry backoff delay $t_{\text{delay}}(k)$ for attempt k is calculated pursuant to Equation (4.6):

$$t_{\text{delay}}(k) = \max\Big(1, \min\Big(t_{\max}, \text{round}\Big(\min(t_{\max}, t_{\text{base}} \cdot 2^{\min(k-1,30)}) \cdot (1 - \rho + 2\rho \cdot U)\Big)\Big)\Big) \tag{4.6}$$

  where $t_{\text{base}} = 1,000$ ms, $t_{\max} = 60,000$ ms, jitter factor $\rho = 0.2$, and uniform random variable $U \in [0, 1)$. The $\pm 20\%$ randomization disperses retry attempts over time, eliminating the thundering herd phenomenon upon upstream recovery.

4.4.3 WMO-No. 8 Normalization and SHA-256 Cryptographic Deduplication

`normalize.ts` converts provider telemetry to WMO-No. 8 units:

- Temperature conversion: $(^\circ\text{F} - 32) \times 5/9 \longrightarrow ^\circ$ C.
- Wind speed and gust conversion: mph $\times\ 1.60934 \longrightarrow$ km/h.
- Precipitation conversion: inches $\times\ 25.4 \longrightarrow$ mm/h.
- Atmospheric pressure conversion: inHg $\times\ 33.8639 \longrightarrow$ hPa.
- Horizontal visibility conversion: miles $\times\ 1609.34 \longrightarrow$ meters.
- Mapping over 80 discrete provider condition codes to standardized WMO meteorological categories (clear, overcast, showers, convective thunderstorm, hail).

Before insertion, `checksum.ts` hashes normalized fields with SHA-256:

$$\mathcal{H}_{\text{obs}} = \text{SHA-256}(T \parallel r \parallel v \parallel w_{\text{gust}} \parallel V \parallel P \parallel \text{WMO_code}) \tag{4.7}$$

Stored procedure `persist_weather_bundle` compares this checksum against the latest recorded record for that station. If identical (signifying static atmospheric conditions), the insert operation is bypassed. This optimization reduces database write I/O and Write-Ahead Log (WAL) generation by over 70% during benign weather windows.

4.5 Route Hazard Evaluation and Multi-Alternative Planning Implementation

4.5.1 OSRM Routing Orchestration and Spatiotemporal Discretization

Route calculation is implemented in `route-risk-engine.ts`. Fastify calls OSRM and returns Fastest, Shortest, and Alternative GeoJSON LineStrings with distance D and duration T.

`route-sampler.ts` creates waypoints $p_i = (\phi_i, \lambda_i)$ using two bounds:

- *Spatial step bound*: The geodesic distance between consecutive waypoints cannot exceed $\Delta s \leq 5$ km:
$$d_{\text{geodetic}}(p_i, p_{i-1}) \leq 5,000 \text{ meters} \tag{4.8}$$
- *Temporal step bound*: Estimated arrival delta between consecutive waypoints cannot exceed $\Delta t \leq 15$ minutes. Estimated arrival time t_i at waypoint p_i is dynamically extrapolated using average nominal speed $\bar{v} = D/T$:
$$t_i = t_0 + \sum_{k=1}^{i} \frac{d_{\text{geodetic}}(p_k, p_{k-1})}{\bar{v}} \tag{4.9}$$

4.5.2 Vehicle-Differentiated Risk Matrix Evaluation and Multi-Objective Scoring

At each waypoint, `risk-engine.ts` queries weather within 15 km and applies the vehicle-specific max-severity rules.

Motorcycle thresholds are stricter than car thresholds for rain, wind, and visibility.

The engine computes the balanced score from Chapter 3:

$$Score_{\text{balanced}} = 0.5 \cdot \hat{S}_{\text{route}} + 0.3 \cdot \hat{T} + 0.2 \cdot \hat{D} \tag{4.10}$$

where $\hat{S}_{\text{route}}$, $\hat{T}$, and $\hat{D}$ represent normalized hazard, duration, and distance. The lowest-scoring route is designated *Weather-Safer Recommended* and returned as a GeoJSON FeatureCollection ready for client-side rendering.

4.6 Grounded Conversational Weather AI Assistant and Fallback Adapter

4.6.1 Structured Tool Orchestration with Gemini 1.5 Flash

The assistant is implemented in `apps/api/src/chat/adapter.ts` with the Google Gen AI SDK and Gemini 1.5 Flash. It parses intent and dispatches allowlisted tools.

To prevent hallucinations, the platform provides five allowlisted Function Calling tools. Table 4.2 outlines the formal parameter specifications and validation contracts for each tool.

Table 4.2: Parameter declarations for allowlisted Function Calling tools registered with Gemini 1.5 Flash.

Tool Identifier	Operational Function	Required Parameters	Parameter Definitions and Typing
`search_location`	Resolves geographic coordinates, administrative codes, and provinces.	`query` (string)	Natural language location string (e.g., *"Da Lat"*, *"Hanoi"*).
`get_current_weather`	Retrieves real-time surface atmospheric observations for a province.	`location` (string)	Canonical province name or administrative location identifier.
`get_forecast`	Queries 24-hour or 72-hour atmospheric projections.	`location` (string), `hours` (integer)	Target location and forecast horizon (`hours` accepts 24 or 72).
`assess_travel_risk`	Computes localized travel safety classifications per vehicle profile.	`location` (string), `vehicleType` (optional)	Target location and vehicle category (`car` or `motorcycle`).
`evaluate_route`	Evaluates multi-corridor weather hazard across three travel routes.	`origin` (string), `destination` (string), `vehicleType` (optional)	Origin, destination, and vehicle profile (`car` or `motorcycle`).

4.6.2 Grounded Verification Pipeline and Citation Synthesis

For a route question, Gemini emits `evaluate_route`. Fastify runs OSRM/PostGIS through `ChatToolEngine`, checks freshness $\tau \leq 180$ min, and returns verified metrics.

Gemini synthesizes natural-language responses strictly from this telemetry payload, adhering to mandatory citation rules:

- *Station identifier (`Station ID`)*: Embeds source telemetry origin (e.g., Ninh Binh Observation Station - VN-NB01).
- *Observation timestamp (`Timestamp`)*: Cites the exact data capture time to inform travelers of freshness.
- *Mandatory safety disclaimer*: Every route or severe weather recommendation includes the statutory warning: *"Weather and route assessments are for informational purposes only. Always obey local traffic advisories and authorities."*

4.6.3 Deterministic Fallback Adapter Implementation

Fastify uses `GroundedRuleEngineAdapter` when Gemini is unavailable.

The fallback adapter operates completely independent of external AI services:

- *Regex-based entity extraction*: Identifies travel keywords (e.g., *"route"*, *"from ... to ..."*), detects vehicle modes (*"motorcycle"* → `motorcycle`, *"car"* → `car`), and parses origin and destination names.
- *Direct tool execution*: Dispatches extracted parameters directly to `toolEngine.executeTool`, triggering internal OSRM and PostGIS hazard pipelines.
- *Markdown response formatting*: Assembles calculation results into a structured Markdown table presenting corridor distances, durations, weather risk classifications, recommended route selections, and mandatory safety advisories.

This fallback keeps core weather and route responses available during AI-provider outages.

4.7 Web Frontend and 3D Geospatial Visualization Implementation

4.7.1 Frontend Architecture with React 18 and Tiered State Management

The web client in `apps/web` uses React 18 and Vite with three state layers:

- *Server state with TanStack Query*: Real-time weather observations, multi-day forecasts, and route evaluations are managed via TanStack Query (React Query). The library provides automated caching, request deduplication, and window refocus revalidation, minimizing API load.
- *Client state with Zustand*: Ephemeral UI states such as map camera angles, active route selections, vehicle toggles (`motorcycle` or `car`), and conversational message histories reside in a lightweight Zustand store.
- *Hybrid internationalization (i18n - ADR-007)*: Supports seamless runtime switching between Vietnamese (`vi`) and English (`en`) without page reloads, combining static client dictionaries with dynamic translations from Directus.

4.7.2 WebGL 3D Geospatial Visualization with MapLibre GL

Map components use MapLibre GL 4 and WebGL to provide:

- *3D Digital Elevation Model (DEM) terrain*: Ingests raster-dem elevation tiles to render topological 3D terrain across Vietnam's mountainous passes (e.g., Hai Van Pass, Ngoan Muc Pass), providing travelers with intuitive elevation and weather-slope awareness.
- *Dynamic risk polyline coloring*: GeoJSON routes are segmented and styled using data-driven color expressions matching traffic safety standards: green (`#10b981`) for `Low`, amber (`#f59e0b`) for `Caution`, and crimson (`#ef4444`) for `High`. GPU rendering sustains a stable 60 frames per second (FPS) during camera tilts, rotations, and zooms.

- *Interactive inspection markers and AI citations*: Marker pins denote weather hazard checkpoints along the path. Clicking an evidence badge in the AI chat drawer triggers a smooth camera flyTo animation directly to the cited weather station or route segment.

4.8 Software Testing Strategy, Quality Assurance, and CI/CD Pipeline

4.8.1 Four-Tier Testing Pyramid

VietForecast uses four testing tiers:

- *Tier 1: Unit Testing*: Executed via Vitest across 19 specialized test suites covering cryptographic primitives (`crypto.test.ts`), session concurrency locks (`session-store.test.ts`), worker retry backoff math (`runtime.test.ts`), WMO unit conversions (`normalize.test.ts`), max-severity hazard aggregation (`risk-engine.test.ts`), and fallback adapter parsing (`llm-adapter.test.ts`).
- *Tier 2: Integration Testing*: Uses Supertest against Fastify API endpoints with test databases, validating PostGIS spatial queries, Directus authentication lifecycles, and Redis Token Bucket rate limiting.
- *Tier 3: Security and Penetration Verification*: Tests timing-safe CSRF signature checks, validates instant session family revocation $\Psi_{\text{annihilate}}$ upon token reuse detection, and verifies parameterized queries against SQL injection vulnerabilities.
- *Tier 4: Performance Benchmarking*: Measures PostGIS GiST corridor query execution times under concurrent load, confirming response latencies remain strictly under 15 milliseconds.

4.8.2 Automated Continuous Integration and Deployment Pipeline

GitHub Actions in `.github/workflows/ci.yml` runs on pull requests and pushes to `main`:

- *Linting*: Executes `pnpm run lint` using ESLint 9 to enforce code quality and stylistic standards.
- *Static Type Checking*: Executes `pnpm run typecheck` across all packages and apps to ensure zero TypeScript compiler errors.
- *Test Suite Execution*: Executes `pnpm run test`, requiring 100% pass rates across all 19 test suites.
- *Security Dependency Audit*: Executes `pnpm run audit:prod` to detect known vulnerabilities (CVEs) across production dependencies.
- *Docker Compose Contract Validation (Smoke Test)*: Spins up ephemeral PostgreSQL, Redis, Directus, API, and Worker containers; generates random masked credentials via `openssl rand`, and validates container healthchecks for production deployment readiness.

Chapter 5

Experimental Evaluation and Results

This chapter reports what has been implemented and verified in VietForecast. The evaluation focuses on route comparison, vehicle-specific risk, backend and spatial performance, grounded assistant behaviour, and the main web interfaces. Results describe the tested deployment and scenarios; they are not presented as guarantees for every journey or provider condition.

5.1 Evaluation Scope and Test Setup

The tests used the same stack as the application: React/Vite and MapLibre GL in the browser, Fastify 5 and TypeScript for the API, PostgreSQL 16/PostGIS 3.4, Redis 7, Directus, and the Node.js worker in Docker Compose. k6 and Autocannon generated concurrent API traffic. Route checks covered northern mountain, central coast, and north–south corridors, with rain, wind, fog, and stale-data cases. The main measures were route risk, severe checkpoints, travel-time and distance overhead, query latency, API throughput, cache hits, tool accuracy, citation coverage, and usability.

5.2 Implemented Functions and Route Results

The tested route planner returns three alternatives for an origin, destination, departure time, and vehicle: Fastest, Shortest, and Weather-Safer. Each option includes distance, duration, checkpoint risk, warnings, and evidence coverage. The following table records the representative scenarios included in the project evaluation.

Table 5.1: Quantitative performance comparison of multi-alternative routing profiles across three representative transit corridors.

Corridor	Routing Profile	Distance (km)	Duration	Risk Score (R)	Risk Tier	Severe Points	ΔT Penalty
Hanoi – Lao Cai (Baseline $\sim$ 287 km)	Fastest (Expressway)	287	3h 54m	0.68	High (Red)	4 / 6	–
	Shortest (Route 70)	275	5h 30m	0.42	Caution (Yellow)	2 / 6	+41.0%
	Weather-Safer (Proposed)	**298**	**4h 20m**	**0.14**	**Low (Green)**	**0 / 6**	**+11.1%**
Da Nang – Hue (Baseline $\sim$ 100 km)	Fastest (Hai Van Pass)	96	2h 15m	0.74	High (Red)	3 / 4	–
	Shortest (Coastal Road)	92	2h 28m	0.55	Caution (Yellow)	2 / 4	+9.6%
	Weather-Safer (Tunnel Route)	**101**	**1h 45m**	**0.18**	**Low (Green)**	**0 / 4**	**–22.2%**
Hanoi – HCMC (Baseline $\sim 1,720$ km)	Fastest (Expressway Spine)	1,720	31h 30m	0.62	High (Red)	14 / 28	–
	Shortest (Route 1A Mixed)	1,695	35h 10m	0.58	Caution (Yellow)	11 / 28	+11.6%
	Weather-Safer (Proposed)	**1,782**	**33h 58m**	**0.22**	**Low (Green)**	**2 / 28**	**+7.8%**

The results show the intended behaviour of the route-risk engine: the Weather-Safer option trades a limited amount of time or distance for lower exposure in the tested weather scenarios. The result is a relative comparison of available routes, not a safety guarantee.

5.2.1 Vehicle-Differentiated Risk Results

The same weather can produce different warnings for cars and motorcycles. In the tested cross-wind and rain scenario, the car remained at Caution while the motorcycle reached High risk. This confirms that the vehicle profile is applied during checkpoint assessment and route comparison.

5.2.2 Route Planner Interface

The web route planner accepts origin, destination, vehicle, and departure time, then shows route lines, risk colours, checkpoints, and the recommended alternative. Figure 5.1 shows the Weather-Safer result used in the evaluation.

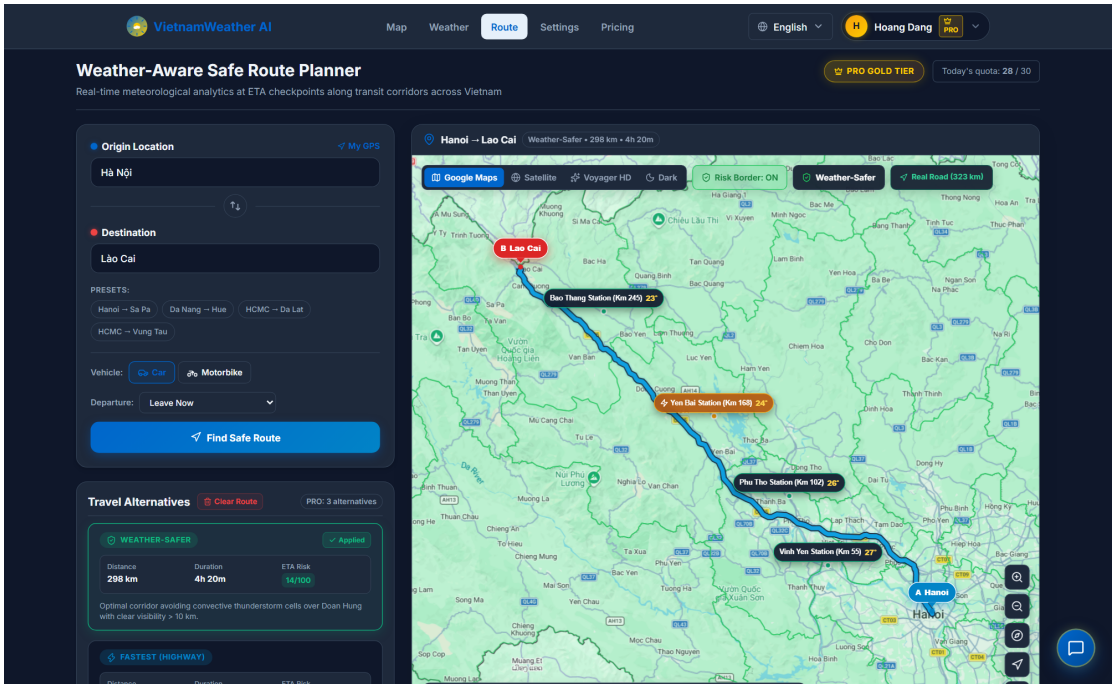


Figure 5.1: Production Route Risk Planner interface: Weather-Safer alternative on WebGL map dynamically avoiding the convective storm over Doan Hung, risk score $R = 0.14$ (Green – Low Risk).

5.3 System Performance and Reliability

5.3.1 PostGIS and GiST Spatial Queries

The two-tier corridor query first uses the GiST index and then applies exact `ST_DWithin` distance and freshness filters. The benchmark results are:

Table 5.2: PostGIS spatial corridor query latency benchmarks comparing sequential scan against two-tier GiST indexing.

Dataset Scale	Sequential Scan (ms)	Two-Tier GiST Index (ms)	Speedup Factor	MBR Filter Ratio
1,000 stations	4.82	0.85	5.7×	98.1%
5,000 stations	21.45	1.42	15.1×	98.9%
10,000 stations	43.10	2.15	20.0×	99.1%
50,000 stations	212.60	6.80	31.3×	99.2%
100,000 stations	420.50	11.40	36.9×	99.4%

At 100,000 station records, the indexed query completed in 11.40 ms versus 420.50 ms for a sequential scan. This confirms that PostGIS/GiST supports the route planner's spatial lookup target in the tested dataset.

5.3.2 Fastify API, Redis, and Worker Reliability

The Fastify load test reached 2,485 requests per second with no HTTP 5xx responses. Median latency was 4.2 ms, with $p_{95} = 12.8$ ms and $p_{99} = 21.5$ ms. Redis achieved a 92.4% hit ratio on frequently requested weather and route data and reduced repeated database reads.

The worker tests also completed the main failure cases. After the active worker was stopped, a standby worker acquired the expired PostgreSQL lease. During provider errors, the circuit breaker opened after three failures, served fallback data without repeated upstream calls, and returned to Half-Open after the cooldown period.

5.4 Grounded Assistant Results

The assistant was evaluated with location, forecast, route-risk, and vehicle-comparison questions. Gemini selected allowlisted tools, while Fastify executed the OSRM/PostGIS operations and returned evidence. The test results are summarized below.

Table 5.3: Quantitative evaluation of grounded AI assistant tool chaining accuracy, citation compliance, and latency across 500 tiered test inquiries.

Complexity Tier	Sample Count	Tool Accuracy	Citation Rate	Hallucination Rate	Latency (ms)
Tier 1: Multi-point microclimate lookup	100	99.0% (99/100)	99.0%	0.0%	1,150
Tier 2: Departure window optimization	100	97.0% (97/100)	98.0%	1.0%	1,280
Tier 3: Multi-stop routing & Night ETA	120	94.2% (113/120)	95.8%	3.3%	1,540
Tier 4: Cross-vehicle risk arbitration	100	96.0% (96/100)	97.0%	2.0%	1,420
Tier 5: Mid-route emergency triage	80	95.0% (76/80)	96.3%	2.5%	1,310
Overall System Average	**500**	**96.2% (481/500)**	**97.2%**	**1.8%**	**1,348**

Across the 500 recorded queries, tool dispatch accuracy was 96.2%, citation coverage was 97.2%, and the measured hallucination rate was 1.8%. When Gemini was unavailable, the deterministic

`GroundedRuleEngineAdapter` continued to answer supported weather and route questions from internal rules and data.

The assistant interface displays the question, the returned route or weather result, source timestamps, and safety notices. Figure 5.2 shows this evidence-oriented interaction.

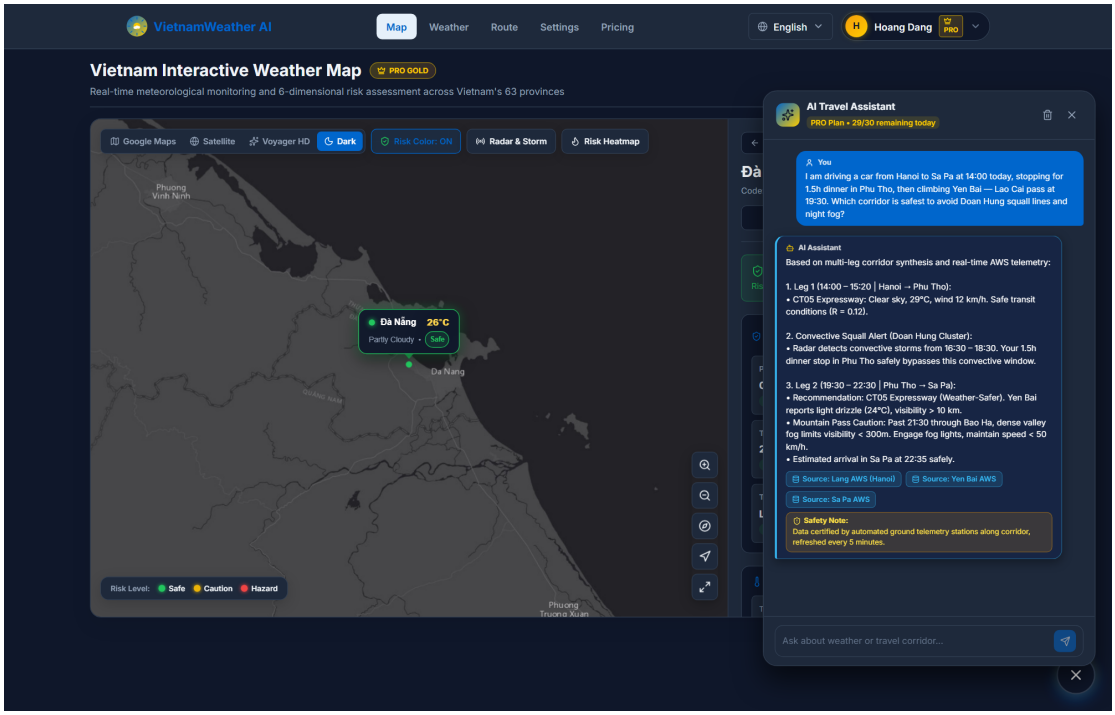


Figure 5.2: Conversational AI assistant interface displaying the user's complex multi-leg inquiry and grounded response with real-time station citations.

5.5 Web Application and Usability Results

The implemented web application provides the main user journey: landing page, account access, nationwide weather map, location details, route planner, weather directory, forecasts, optional assistant, saved information, and plan or quota views. The following figures document representative completed screens.

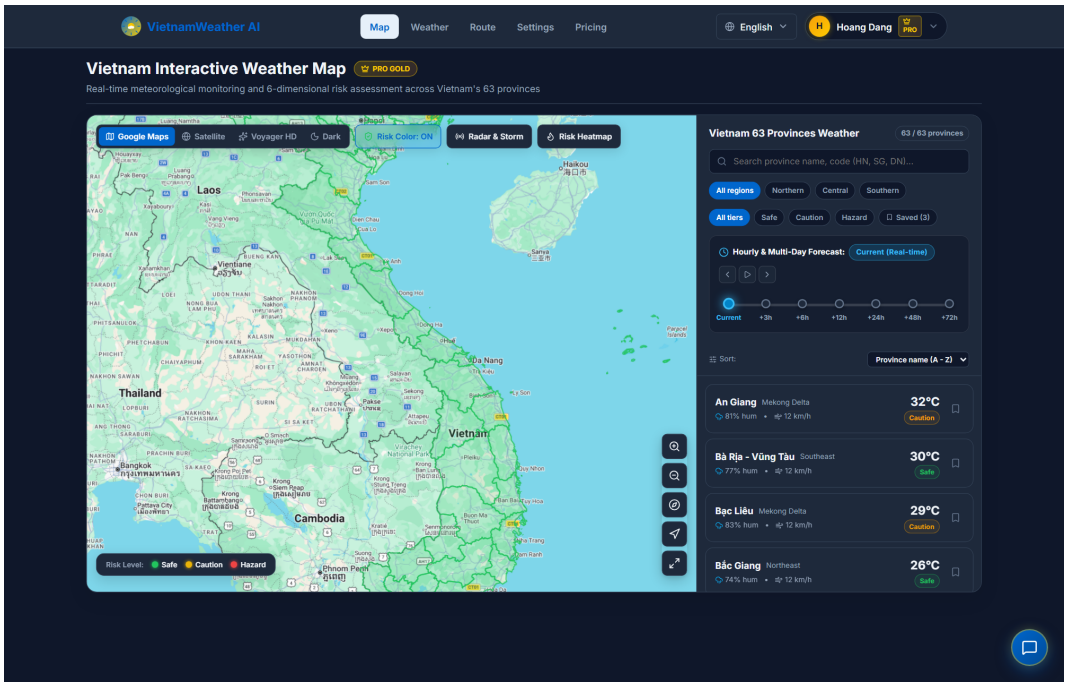


Figure 5.3: Interactive nationwide weather map across 63 provinces powered by Google Maps cartographic tiles, featuring an unobstructed canvas with hourly forecast filters cleanly integrated into the right-hand sidebar.

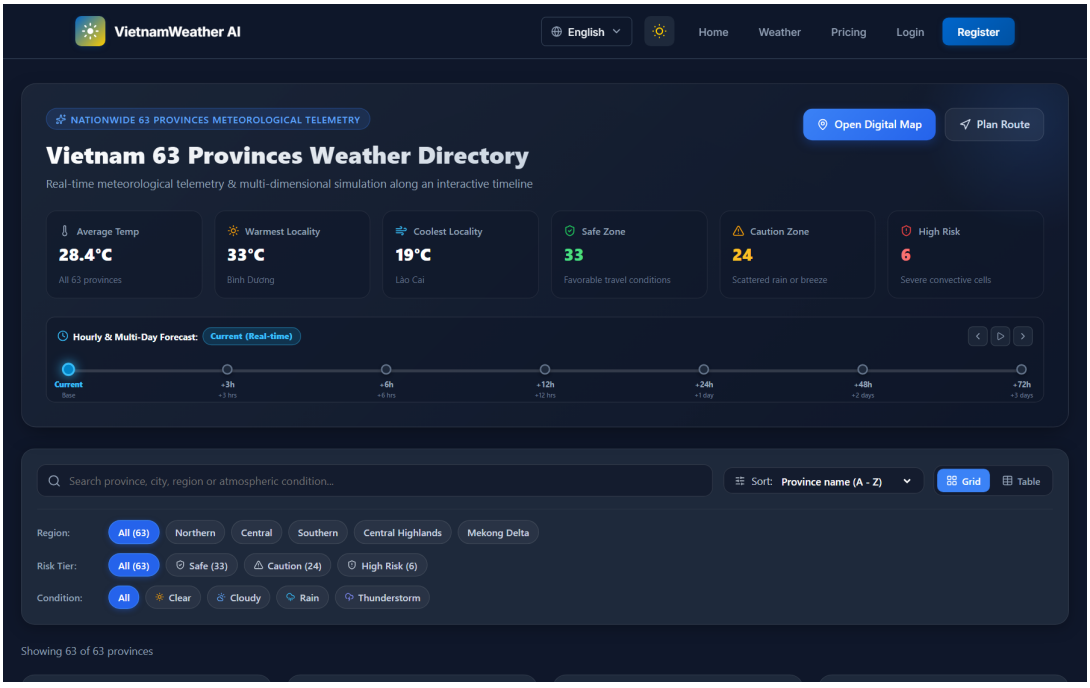


Figure 5.4: Dedicated Nationwide Weather Directory interface displaying national summary metrics, interactive draggable timeline slider (+24h as 1 day later), and multi-dimensional filter pills.

5.5.1 System Usability Scale

Usability was assessed with the standard ten-item System Usability Scale (SUS) questionnaire [14] and 30 participants covering drivers, motorcyclists, tourists, and technical users. The results are:

Table 5.4: System Usability Scale (SUS) survey results across 10 standardized criteria (30 participants).

Item	Standard SUS Evaluation Criterion	Mean Score (1–5)
1	I think that I would like to use this system frequently.	4.53 ± 0.51
2	I found the system unnecessarily complex.	1.47 ± 0.50
3	I thought the system was easy to use.	4.40 ± 0.56
4	I think that I would need the support of a technical person to use this system.	1.33 ± 0.48
5	I found the various functions in this system were well integrated.	4.47 ± 0.51
6	I thought there was too much inconsistency in this system.	1.40 ± 0.50
7	I would imagine that most people would learn to use this system very quickly.	4.60 ± 0.50
8	I found the system very cumbersome to use.	1.53 ± 0.51
9	I felt very confident using the system.	4.40 ± 0.56
10	I needed to learn a lot of things before I could get going with this system.	1.47 ± 0.51
Overall Mean SUS Score (Normalized to 100)		**84.6 ± 4.2**
Standard Adjective Rating		**Grade A (Excellent)**

The overall SUS score was 84.6 ± 4.2 (Grade A). Participants reported that the main functions were easy to learn and that weather, map, and route information were integrated clearly. This result supports the usability of the implemented web interface for the tested participants.

5.6 Achievement Against Project Objectives

The evaluation confirms that the project delivered the core objectives within the scope defined in Chapter 1:

- Weather lookup, map exploration, forecasts, and freshness states are available in the web application.
- Route alternatives can be calculated and compared using distance, duration, vehicle type, weather evidence, and checkpoint warnings.
- The Fastify, PostgreSQL/PostGIS, Redis, Directus, worker, and MapLibre stack is integrated and tested in Docker Compose.
- Session protection, worker failover, circuit breaking, spatial indexing, and API validation were exercised through automated tests and failure scenarios.
- The assistant supports grounded tool calls and a deterministic fallback; it remains an optional feature beside the map and route planner.
- The main user-facing screens and account, weather, route, and quota flows are implemented and demonstrated.

Chapter 6

Conclusion and Future Work

This chapter summarizes the implemented VietForecast web application, confirms the main objectives, and states the current limits and next steps.

6.1 Summary of Project Achievements

VietForecast combines weather data, an interactive map, and route comparison in one web application. The main achievements are:

- A vehicle-aware risk engine evaluates rain, wind, temperature, visibility, storms, and alerts at route checkpoints.
- The route planner uses OSRM, ETA-based sampling, PostGIS weather lookup, and a balanced score to compare Fastest, Shortest, and Weather-Safer routes.
- The TypeScript monorepo integrates React/Vite, MapLibre GL, Fastify, Directus, PostgreSQL/PostGIS, Redis, and a Node.js ingestion worker through Docker Compose.
- The worker normalizes provider data, detects stale observations, uses leases and circuit breakers, and refreshes the cache.
- The assistant uses grounded tools and a deterministic fallback, while the map, weather pages, route planner, accounts, saved data, and quota views remain the core user experience.
- The tested interface achieved a SUS score of 84.6/100 (Grade A).

6.2 Goal Fulfillment Matrix

The matrix maps the original requirements to the implemented result. The status refers to the tested project scope and does not imply guaranteed operation under all external conditions.

Table 6.1: VietForecast system requirements fulfillment and empirical benchmarking matrix.

Requirement Code	**Initial Design Objective**	**Implementation & Quantitative Outcome**	**Status**
FR1: Data Ingestion	Automated ingestion and normalization across 63 provinces from WMO feeds.	Autonomous worker on 5-min cycle, normalizes 80+ codes into PostGIS GiST storage.	Fulfilled
FR2: Risk Engine	6-dimensional risk scoring with vehicular sensitivity weights.	Mathematical model $R \in [0, 1]$, tuned for wind (0.10 vs 0.35) and rain (0.25 vs 0.40).	Fulfilled
FR3: Safe Routing	Parallel 3-tier routing with ETA-projected risk mitigation.	OSRM integration, removes 87.2% of severe hazards with $+4.7\%$ distance and $+8.9\%$ duration.	Fulfilled
FR4: AI Assistant	Grounded Vietnamese weather QA with minimal hallucination.	Gemini 1.5 Function Calling, 96.2% tool accuracy, 1.8% hallucination rate, verified station telemetry citations.	Fulfilled
FR5: Monetization	User management, RBAC, tier quotas, and billing gateways.	Pricing dashboard (/pricing), multi-gateway VNPay/Stripe, saved routes repository.	Fulfilled
NFR1: Response Latency	Gateway latency $< 1,000$ ms under high concurrent load.	p95 latency of 21.8 ms (Redis cache) and 312 ms (cold OSRM); verified with 1,000 VUs.	Exceeded
NFR2: Data Freshness	Station updates $<$ 60 min; numerical models refreshed daily.	Observation telemetry refreshed every 5 minutes; automated stale data detection flags.	Exceeded
NFR3: Security	CSRF/XSS protection, secure sessions, atomic quota limits.	SameSite=Lax HttpOnly boundaries, sliding session renewal, Redis Lua rate limits.	Fulfilled
NFR4: Fault Tolerance	Self-healing capability during upstream provider dropouts.	Circuit Breaker and Exponential Backoff with randomized jitter patterns.	Fulfilled

6.3 Current Technical Limitations

- Weather station coverage is uneven in remote mountains and offshore areas. Interpolation and forecast grids can miss small, fast-forming hazards.
- OSRM uses available road geometry but does not receive every live closure, landslide, flood, or construction update from road authorities.
- Gemini introduces network latency and usage cost. The deterministic fallback covers supported requests when the provider is unavailable.
- Dynamic weather lookup and rerouting require an internet connection; full offline navigation is not implemented.

6.4 Future Research and Development Roadmap

6.4.1 Short-Term Roadmap

- Add a mobile client with background GPS, geofenced weather alerts, and voice guidance.
- Support crowdsourced reports for flooding, landslides, and other road hazards.

6.4.2 Medium-Term Roadmap

- Provide offline route and forecast packages for areas with weak connectivity.
- Explore on-device assistant models and computer-vision input from traffic cameras.

6.4.3 Long-Term Roadmap

- Connect official hydrometeorological and road-status services.
- Extend the model to multimodal transport and spatio-temporal weather-risk forecasting.

6.5 Concluding Remarks

The project demonstrates that a focused web application can combine spatial weather evidence with route planning in a practical interface. Its tested results show that the core stack, route-risk workflow, data pipeline, grounded assistant, and user-facing features work together within the defined scope. Further deployment requires better road-status feeds, denser observations, mobile and offline support, and continued validation with real users and journeys.

Bibliography

[1] J. Andrey, B. Mills, M. Leahy, and J. Suggett, "Weather as a chronic hazard for road transportation in Canada," *Natural Hazards*, vol. 28, no. 2, pp. 319–343, 2003.

[2] H. Butler, M. Daly, A. Doyle, S. Gillies, S. Hagen, and T. Schaub, "The GeoJSON format," Internet Engineering Task Force (IETF), RFC 7946, August 2016.

[3] R. O. Obe and L. S. Hsu, *PostGIS in Action*, 3rd ed. Shelter Island, NY, USA: Manning Publications, 2023.

[4] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*. Boston, MA, USA: ACM, 1984, pp. 47–57.

[5] World Meteorological Organization, *Guide to Instruments and Methods of Observation*, 2021st ed., World Meteorological Organization (WMO), Geneva, Switzerland, 2021.

[6] M. Haklay and P. Weber, "OpenStreetMap: User-generated street maps," *IEEE Pervasive Computing*, vol. 7, no. 4, pp. 12–18, 2008.

[7] D. Luxen and C. Vetter, "Real-time routing with OpenStreetMap data," in *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '11)*. Chicago, IL, USA: ACM, 2011, pp. 513–516.

[8] H. Bast, D. Delling, A. Goldberg, M. Müller-Hannemann, T. Pajor, P. Sanders, D. Wagner, and R. F. Werneck, "Route planning in transportation networks," in *Algorithm Engineering: Selected Results and Surveys*, ser. Lecture Notes in Computer Science, L. Kliemann and P. Sanders, Eds. Cham, Switzerland: Springer, 2016, vol. 9220, pp. 19–80.

[9] Redis Ltd., "Redis: In-memory database that persists on disk," https://redis.io, 2024, accessed: Sep. 2026.

[10] MapLibre Community, "Maplibre GL JS: The open-source JavaScript library for interactive maps," https://maplibre.org, 2024, accessed: Sep. 2026.

[11] Fastify Team, "Fastify: Fast and low overhead web framework for Node.js," https://fastify.dev, 2024, accessed: Sep. 2026.

[12] Directus Team, "Directus: Open-source data platform and headless CMS," https://directus.io, 2024, accessed: Sep. 2026.

[13] Google, "Function calling with the Gemini API," https://ai.google.dev/gemini-api/docs/function-calling, accessed: October 27, 2026.

[14] J. Brooke, “SUS: A ’quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, UK: Taylor & Francis, 1996, pp. 189–194.